

编码 Agent 项目面试题库

编码 Agent 项目面试题库（逐章 + 全局）

使用说明：每章按“面试官真会问的问题”筛选过，不含任何代码细节（变量名/函数名/类名）。答案按“可口述”的密度写：机制 + 为什么 + 否掉了什么备选。【追问】是压力面的典型连环问，答案已并入正文或单独给出。

第〇部分：全局题（必考）

Q0.1 两分钟介绍一下这个项目

答：我用 Python 从零实现了一个结构和 OpenAI Codex 同构的编码 Agent。它是一个能读文件、跑 shell、改代码的对话式程序，核心是一个“模型循环”：模型每轮可以输出文本或工具调用，框架执行工具、把结果追加回历史、再喂给模型，直到模型不再要工具为止。在这个最小循环之上，我按真实系统的顺序逐层加了：供应商协议归一化、shell 工具的进程管理、基于上下文锚定的文件编辑、命令审批与沙箱、上下文压缩、崩溃恢复、工具并发调度、MCP 外部工具接入、子 Agent、任务计划（待办清单）、重试与错误分类、系统提示词分层与缓存、评估与回归体系、跨会话记忆、技能（渐进式披露）、Web 控制台、内核级沙箱。整个项目有 1800+ 个测试，跑在可回放的“假模型”上，不依赖真实 API key。

追问：哪一块你觉得最难？ 参考答案：不是单个模块，而是“静默错误”——全书 218 条故障里 89% 不报错，表现为一个看起来合理的错误答案。对抗它们的方法论（录制回放、变异测试、A/B 实验）比任何单个功能都值钱。

Q0.2 整体架构怎么分层的？为什么这么分？

答：核心分层是：①模型客户端层（处理 SSE 流、供应商方言归一化）；②Agent 循环层（纯逻辑，不认识任何具体工具和供应商）；③工具层（shell、文件编辑、子 Agent 等各自独立）；④装配层（整个程序只有一个地方知道“怎么把零件拼成一次运行”）；⑤守卫层（审批、调度、沙箱横切在执行路径上）。分层的规则只有一条：**依赖只朝下，上层通过注入拿到下层**。判断某段代码该放哪层的试金石是“改供应商、换工具、换 UI 时，改动被关在哪个边界内”——Web 控制台那一章做到了前端整套换掉而 Agent 核心零行改动，就是这条规则的兑现。

追问：为什么不做成插件式架构？ 因为插件化的前提是有第二个实现，而我每个位置都只有一个实现。我遵循“三次法则”：同一个形状出现三次才抽象，错的抽象比没有抽象更难拆——后来的代码会把它当约束一层层长上去。

Q0.3 说说你遇到的最难的 bug

答（推荐讲两个之一）：

故事 A（两个都成功的编辑，一个消失了）：模型一轮里发两个编辑同一文件的调用，两个都返回“成功”，文件里却只剩一个修改。根因是并发下两个调用都读到编辑前内容、各自计算、后写者覆盖先写者。最反直觉的测量结果是：把人为延迟从 50 毫秒降到 0，丢失率不变（30/30）——说明这不是竞态窗口宽的问题，是这条路径从来没被并发测试过。修法是给每个工具声明“资源足迹”（读哪些路径/写哪些路径/说不清楚就标“有状态”），调度器按足迹分批，冲突的调用绝不重叠。更深一层：说不出足迹的默认当“有状态”（与一切冲突、串行），这个保守默认后来真的把一次遗漏从“数据丢失”降级成了“变慢”。

故事 B（竞态不一定是 flaky 的）：多个进程消费同一批任务时出现重复认领。大家默认竞态是偶发的，我实测四次运行四次都重复——病根不在写的那几行，而在没写的那一行：SQLite 默认在第一条写语句才开事务，SELECT 不算写，所以“先查再改”的查根本跑在事务外。修法是把显式立即事务放到查询之前。这个案例我常用来反驳“竞态没法稳定测试”。

Q0.4 你的项目里 89% 的故障是静默的，怎么发现它们？

答：先分类，静默故障有几种典型形态：①协议静默（流式分片只带一部分字段，不崩但行为错）；②数据丢失静默（并发覆盖、双双报成功）；③机制从未运行（代码正确但输入从来没来过）；④安全静默（绕过审批的表现就是“命令正常运行”）；⑤成本静默（缓存失效，只是账单变贵）；⑥测量工具自己报“关于别的东西的数字”。对应的发现手段：**录制回放**（把真实响应逐字节录下来重放，杜绝运气性通过）、**变异测试**（把修复改回去看测试红不红——这是立功最多的手段，多个“测试名声称的比断言多”的假测试全是它抓的）、**边界测试**（故意喂极端输入）、**双供应商 A/B**（很多“复现”只在一侧成立）、**可观测性**（读落盘的对话实录）。核心态度：不能等静默错误自己报告，必须主动去撞，而主动撞需要在需要它之前就建好的工具。

Q0.5 如果重新做一遍，你会改什么？

答：三个真实的反思：①**跨章节知识不传播**——同一个正确答案（比如环境变量白名单、超时异常的写法）在仓库里存在两份，其中一份是对的，另一处照样错了很多章。后来我学会“两个必须说同一件事又不能互相 import 的地方，写一个比较它们的测试”。②**先查再抄**：记忆系统第一版我是按“常识”设计的，后来打开被复刻系统的源码，发现四处机制全都和它实际做法不同，而且每一处都不是有意识的取舍，是“没查”。查了之后有意偏离和没查自己瞎猜，是两种完全不同的东西。③测量工具要预算它自己 1-2 个 bug——我连续多章验证“每个测量工具的故障形状永远是报告一个关于别的东西的数字”。

第一部分：工程化起点（打包与 CI）

Q1.1 为什么从打包和工程化开始，而不是先写 Agent 功能？

答：因为后面每一步都要回答“我改的东西到底生效没有”。如果代码装不到一个确定的地方、测试跑的不是交付物，那所有验证都在测一个幻觉。表面上这一步产出的是“能安装的包”，真正产出的是三个能替我发现问题的基础设施：一个不给“白送 import”的目录结构、一个会打开安装包检查内容的测试、一台不是我的电脑（CI）。

追问：什么是“白送 import”？Python 的搜索路径第一项是当前目录，如果仓库根目录里正好有个和包同名的文件夹，**不安装也能 import 成功**——这让所有打包错误静默隐身。用 src 布局（源码放进 src 子目录）之后，不安装测试直接启动失败。本质不是“更整洁”，是**把一个悄悄出错的场景改造成大声报错的场景**。

Q1.2 测试全绿，但用户装上就是坏的——怎么防？

答：开发用的是可编辑安装（源码树直接挂进路径），它对打包问题**完全免疫**——包括对 bug 免疫。所以光跑单元测试永远绿。我加了两道：①断言“已安装”这件事本身（查包元数据，源码目录给不出来的东西）；②把 wheel 构建出来、解开、检查数据文件真的在里面——0.3 秒的测试，是整个套件里唯一能抓住“数据文件没进包”这类问题的。

追问：怎么知道这个测试真的有效？变异验证：写完修复，把修复改回去，确认对应测试会红。**对着坏代码也能通过的测试不是回归测试，是装饰品。**

Q1.3 CI 里放哪些检查？放多少？

答：我的标准是“一个检查要能挡住合并的 CI，前提是它拦下过真实问题”。反例是“最佳实践全家桶”：项目还没功能 CI 已经 18 分钟，间歇性红叉教所有人看见红直接合并——被绕过的检查保护效果精确为零，而且比没有更糟，因为它给了安全感。我的 blocking CI 一直控制在六步以内；重的检查（变异测试、flake 探针、真模型评估）分层放到合并后和每夜的 workflow。判断“这一步凭什么挡合并”——答不上来的就去别的层。

第二部分：最小 Agent 循环

Q2.1 描述一下最小 Agent 循环的状态机

答：初始化历史（放用户消息）→ 循环：把**完整历史**全量发给模型（模型无状态，“上下文”就是重发的这个列表）→ 收到一轮响应（文本 + 可能有工具调用）→ 把助手消息追加进历史 → 如果没有工具调用就结束（任务完成），否则逐个执行工具、把结果作为工具消息追加 → 重复，直到轮次上限。有两个关键细节：轮

次快耗尽时提前注入一条系统注记“你还剩 N 轮，收尾吧”——**预算必须让模型看得见**；上限是可配置的，测试里可以调小——能被测试调小的上限比写死的常量好测得多。

Q2.2 终止条件为什么是“没有工具调用”而不是“有文本输出”？

答：这是我踩过的重要的坑之一。系统提示要求模型“调工具前先说一句话”（真实编码 Agent 几乎都这么要求），所以模型会在同一轮里先旁白再调工具。如果“有文本就停”，程序会在旁白处收工，返回一句语法正确、语气笃定的**自信的非答案**，退出码还是 0。这个错误只有把“模型说的”和“模型做的”对比才能发现——由此我得到一条贯穿整个项目的原则：**永远不要相信模型对自己行为的描述，断言可观测的副作用。**

Q2.3 工具的错误为什么返回字符串而不抛异常？

答：站在模型角度想：工具失败是一条**信息**——模型读到“文件不存在”有能力换个路径重试；进程崩溃才是灾难——模型没有能力处理“你已经死了”。所以工具执行层永远返回文本、永不抛异常，错误信息按三段式写：出了什么错 / 你能用什么 / 下一步做什么。这条规则后来覆盖了三种失败：工具名不存在、参数不是合法 JSON、工具内部异常。**工具的错误信息就是 prompt**——只说“什么坏了”的错误会让模型原样重试，说了“该怎么办”的错误让它一次就对。我实测过：裸异常文本下弱模型三次全部原样重试同一个错误路径，三段式信息下全部一次改对。

Q2.4 不依赖真实 API，怎么测试 Agent？

答：四层组合。①模型抽象是结构化协议（Protocol），测试可注入任意假实现；②**录音器**把每次跨模型边界的请求/响应全量落盘（追加式 JSONL、每行写完即强制刷盘、键名脱敏、永不向调用方抛异常——一个能把被调试程序搞崩的调试工具比没有更糟）；③**桩服务**把真实响应逐字节回放，控制键用特殊前缀藏在请求体里选择播哪段录音、在哪截断流，真服务器会忽略不认识的键，生产零成本；④关键决策：**走真 socket 而不是 mock HTTP 客户端**——要测的恰恰是流终止哨兵不是 JSON、前缀协议这些解析层的坑，mock 掉客户端就全绕过去了，剩下的测试只能证明“我的假数据和我的解析器互相同意”。代价是诚实的：录音会过期（供应商换协议后测试全绿，因为它们在和化石对话），需要定期重录。

追问：为什么真实模型“大部分时候输出相同”反而更危险？因为你会把“这次对了”当成“修好了”。非确定性不是靠多跑几次对冲，而是把那次真实响应逐字节录下来、逐字节播回去。

Q2.5 事件系统怎么设计的？

答：信任边界处立一道翻译层：服务端的任意 JSON 到解析模块为止，往外走的是自定义事件类型。只有三种：文本碎片、完整工具调用、完成信号。为什么是三个类而不是一个带可选字段的类——因为一个类允许构造出“既有文本又是终止信号”这种无意义组合，三个类让**非法状态无法表示**。事件全部冻结（不可变），因为它们是“已经发生的事实”，顺带换来可比较、可做字典键、测试直接断言相等。这三个事件不是设计出来的，是先写了三十行“把原始流打印出来”观察出来的：文本按任意长度碎片来（路径会被劈成两半，拼完前不能做任何判断）、工具调用一次性给完、终止有两个信号（模型为什么停 + HTTP 流结束了）——**没有终止哨兵的流不是“说完了”是“被切断了”，整轮作废。**

Q2.6 异步里的经典坑你踩过哪些？

答：①忘了 `await`：不报错，只构造协程就丢弃，唯一信号是垃圾回收时的警告——我把这个警告升级成测试错误；②事件循环里的阻塞 IO 会把所有并发悄悄退化成排队——所以阻塞的文件读取丢给线程池；③宽泛的 `except Exception` 拦不住取消异常（它继承自更底层的基类）——取消会直接穿透，留下“发起了工具调用但没有结果”的坏状态，这个我在中断恢复那块专门处理了。

第三部分：协议层

Q3.1 为什么“协议先行”？归一化层放在哪？

答：我的做法是**先测量再设计**：不读文档，拿同一段代码问两家供应商、把原始流打出来对比。三个实测事实：一家把一次工具调用切成十几个分片（只有第一个带名字和 id），另一家一个分片给完；两家的历史“方言”不同（参数一个是 JSON 字符串一个是对象，结果配对一个用调用 id 一个用工具名）；**只有一部分服务端检查不变量**——同一份坏历史，宽松的那家全放行返回 200 加胡言乱语，严格的那家全 400。归一化层放在模型客户端模块：分片在这里攒完、分歧在这里止步，上层代码永远只见一种形状——三个分片和十四个分片，上层看到的产出完全相同。这个决定的红利在换供应商时显形：Agent 循环一行没动。如果先设计再测量，最可能的产物是一个“通用适配器基类”，然后发现真实差异根本不在你设计的维度上。

Q3.2 历史里存什么？为什么不直接存服务端的 JSON？

答：存“发生了什么”的事实——一次对话只有四种：用户说的话、助手的话（含工具调用）、工具结果、系统注记。发送时才按各家方言翻译。理由：同一段对话两家要的形状不一样，存了方言，换供应商旧会话就废了。还有一个容易漏的点：**模型原始发的参数字符串要原样保留一份**——回发时必须是它原本发的字节，重新序列化会改变空白和键序，某些供应商认不出。这份原始数据最初是为了错误信息（告诉模型“你发的不是合法 JSON”必须回显它实际发了什么），后来在恢复、回放里反复兑现——为说清错误多留一份原始数据，是个反复有用的模式。

Q3.3 “每个工具调用必须被恰好应答一次”——这个不变量怎么强制？

答：做成历史容器的入口检查，而不是出口检查或调用点约定。容器内部维护一个“未应答账本”：追加助手消息时登记它发起的调用，应答时从账本取出；追加下一条助手消息前如果还有欠账就拒绝；重复 id 拒绝；应答一个不存在的 id 就报错，且**报错说出它在等哪些 id**。为什么放入口不放循环里——因为后来出现了压缩后重发、崩溃恢复后重发、子 Agent 等多条路径，每条路径都可能忘校验，但每条都必须经过“往历史里放消息”这扇门。**把检查放在所有路径的交汇点，通常就是数据离开你控制范围的那一刻**。配套实测：宽松供应商会带着坏不变量一路绿灯让你把 bug 带上线——服务端不替你守，必须自己守。

Q3.4 架构规则怎么保证不被后来的人破坏？

答：口头约定的架构规则 100% 会烂掉，所以写成可执行测试。两条例子：上层模块**不许 import HTTP 客户端、不许出现任何供应商名字**——如果有人图省事在循环里写“如果是某家供应商就特殊处理”，功能还是对的、没有任何测试变红，但归一化就白做了；共享类型所在的叶子模块必须保持是叶子。检查用 AST 解析而不是字符串匹配（注释里的 import 不该算违规）。还有一个细节：测循环 import 必须开子进程——循环 import 只在第一次导入时炸，模块进了缓存后再 import 就直接取缓存，同进程里其他测试早已导入过，这个测试**对着真正循环的代码库也会绿**——测试逻辑完全正确，错的是运行环境。

追问：循环依赖怎么解？先诊断：循环依赖是个信号不是麻烦，它在说“这两个模块共享了某个概念而这个概念没有自己的位置”。共享的是**名词**（类型、常量）就下沉到谁都不依赖的叶子模块；共享的是**动词**（行为、函数）才需要依赖倒置——把行为由构造方递进来。函数内延迟 import 是最差的解法：它只是把失败时刻从启动推迟到第一次调用，而且会骗过只读文件头的检查器。

第四部分：Shell 工具

Q4.1 超时为什么要杀整个进程组？

答：如果只杀手里那个进程 ID，杀掉的是 shell 本身，真正干活的命令是 shell fork 出来的孙进程，会被系统收养**继续跑**。只要命令里有管道、分号、逻辑与，干活者就永远不是你手里的 PID——这是用 shell 执行的结构性问题，不是边界情况。正确做法是让每条命令启动时自成一个新进程组，超时后对整组发不可捕获的强杀信号。我验证资源清理时学到的教训：**必须直接检查系统状态（进程表），不能只验证调用方看到了什么**——“超时异常抛了、错误信息返回了，但进程还活着”这个 bug 就是这样被发现的。

Q4.2 输出怎么截断？为什么要保留头尾两半？

答：先说读取策略：增量读取加硬上限，而不是“跑完再截”——实测上百 MB 输出会把 Agent 进程内存涨好几倍，因为全量捕获机制会把子进程输出完整攒进内存。再说截断形状：保留头尾各一半、中间替换为省略说明。依据是大多数命令行工具的输出习惯：“开头说错了什么，中间是过程，结尾是结论”。我用真实形状的测试输出验证过：失败原因在最前面、摘要在最后、中间是大量通过的记录——只留尾巴会把错误原因整个丢掉。这是典型的**静默故障**：程序完全正确地运行、正确地截断，产出却对模型毫无用处。

追问：截断按字符还是按字节？按字符，且发生在已解码的字符串上；非法字节用替换字符而不是抛异常——保证二进制垃圾不会让整个工具调用崩溃。

Q4.3 cd 怎么处理？

答：在框架层拦截解释，而不是交给 shell。因为 cd 在一次调用的子 shell 里生效，下次调用就消失了；而给子进程传“初始工作目录”参数只能控制从哪开始，无法知道 cd 之后去了哪——cd 是 shell 内建命令，它对 shell 进程状态的修改从不传回调用者。所以识别 cd 命令、自己算目标目录、更新会话状态。明确的边界：只处理 cd 作为独立命令，不处理复合命令里的 cd。**真正的持久 shell（长期存活的 shell 进程）是另一**

个量级的机制，刻意不做——提前做只会引入当前不需要的复杂度。这里有个设计转变值得说：工具从无状态的纯函数（读文件）演化成有状态的会话对象——工作目录和环境变量要在多次调用间存活，而**会话状态属于一次对话，不属于进程**：两个 Agent 不能共享同一个 shell 视图，否则会看见彼此的 cd。

Q4.4 环境变量为什么用白名单？

答：动机是一次回顾发现的泄漏链：程序把 API key 读进了进程环境，子进程默认继承全部环境变量——模型让 Agent 跑的任何命令（包括被提示注入影响的命令）都能看到它。黑名单必然漏（你猜不全哪些危险），白名单默认拒绝一切、只放行命令行工具基本需要的几个变量；将来要放行谁，是一次显式的、code review 里看得见的改动。这条原则我在审批系统里又用了一次：**黑名单的上限是写它的人的想象力，而攻击面的下限是别人的想象力**。

Q4.5 后台命令为什么不支持？

答：实测发现 shell 立刻返回但后台进程继续跑，而且完全游离在输出捕获、退出码、进程清理所有机制之外。真正支持它需要一整套任务注册表、轮询接口、会话结束清理——是一个完整功能而非两行修复。**拒绝一个功能，比默默实现一个有漏洞的版本更诚实**——假装支持的代价是进程持续泄漏，而且用户要到真正用它的那天才发现。

第五部分：工具描述工程

Q5.1 工具描述和 API 文档的区别是什么？

答：模型看到的不是你的函数，是那段 JSON 描述。这段文字是 **prompt 不是文档**：每一个词都参与一次推理，而且**每一轮请求都要重发、都要付费**。由此推出两个工程要求：①描述里的数字必须从代码常量生成、并有快照测试钉住——我实测过模型真的会把描述里的默认值原样传回来，一旦代码改了描述忘改，模型会一直用旧值覆盖新默认，**而且这永远不会报错**；②描述的任何改动都是行为改动——我测过只换描述里例子里的文件名，两家模型从全对翻转为全错。所以描述被抄进测试，改它必须过快照这道门。

Q5.2 描述里写约束有用吗？

答：我做过一组对照实验，结论非常清晰：**描述能改变模型“选哪个工具、输出什么形状”（都在模型控制之内），不能改变模型“对外部世界的预期”**。最强证据：在描述里写“命令超过 30 秒会被杀”，六次采样全部照样发出两分钟的命令——要求模型改变外部世界的预期（它没法让测试跑得更快），零效果；而在错误信息里给指令（只要求模型改变下一步输出的形状），对弱模型是决定性的。

Q5.3 “不要用这个工具，用那个”——什么时候需要写？

答：它是一个**消歧工具**，不是通用补丁。复现条件很精确：两个名字含糊、功能重叠的工具，模型全部选错；各加一句“不要用于 X，用那个”后全部改对。但名字自明的两个工具（读文件 vs 跑命令），不加也全对——名字本身已经完成了消歧。给每个工具都加是在浪费 token，而描述每轮都要重发。这本身就是个好考点：**“业界最佳实践”被实测限定成了条件性规则。**

Q5.4 模型给的路径不对，在描述里写清楚规则有用吗？

答：没用，而且有决定性实验：固定描述规则文字不变、只换例子中的文件名，两家模型从全对翻转为全错——**模型是照抄例子，不是理解规则。**所以路径问题从描述层搬到代码层做确定性处理：仓库内的绝对路径无歧义直接接受、仓库外的拒绝、裸文件名恰好唯一匹配到仓库里一个文件时直接在错误信息里告诉它是哪个。三种“找不到”对应三种下一步：唯一候选直接给出、多候选列出让模型挑、零候选教它用什么命令去找。原则：**能在确定性代码里解决的，绝不推给模型。**模型负责意图，代码负责正确性。

第六部分：文件编辑 (apply_patch)

Q6.1 为什么用上下文锚定而不是行号？

答：这是实测决定的，不是偏好。两个候选：unified diff（行号 + 上下文，git 用的标准格式）和上下文锚定（贴一段原文、说替换成什么）。实验：要求 unified diff 时，一家三次里错一次且错得彻底（声称第 20 行实际在第 17 行，照着切会切到别的代码），另一家两次干脆不理睬要求、自发输出了另一种无行号格式——一个被大量补丁数据训练过的模型自己选的代表法里没有算术，**这是对“别让模型数行”最有力的一票，而且不是我投的。**要求上下文锚定时，两家六次全部精确匹配一字不差。结论：你要模型做的事得是它擅长的事——复制原文它擅长，数行它不擅长。

Q6.2 匹配到多个位置怎么办？

答：**歧义是错误，永远不是猜测。**朴素实现的字符串替换会把所有匹配处全部替换——改一处落地三处，还报告成功。而通用代码块（空值检查、return 语句）在一个文件里出现两三次是常态。没有任何信息能告诉我们模型指的是哪一个；挑第一个、最像的，都是在替模型做一个它没做的决定，而且不会有人发现。正确行为是拒绝并报出所有匹配位置的行号——模型才知道该往哪边扩大上下文范围让锚点唯一。配套的描述写法也验证过有效：“锚点必须恰好出现一次，不唯一就包更多上下文行直到唯一”——它约束的是模型输出的形状，所以模型做得到。

Q6.3 模糊匹配怎么设计？

答：三级匹配、逐级放宽、**每级必须唯一**：精确匹配 → 忽略行尾空白 → 忽略缩进。放宽的方向严格对应实测到的模型抄写误差形态：模型贴回的原文要么逐字节相同，要么尾随空白被规整化，要么从嵌套上下文抄出

时被重新缩进。顺序看起来显然但“显然不等于被测过”——变异测试把顺序倒过来，当时的全部测试仍然全绿，说明顺序从未被测过。还有个反直觉发现：精确子串匹配在嵌套缩进面前**自己就有歧义**（八空格缩进的行包含四空格锚点作为子串），所以“精确匹配肯定唯一”的直觉是错的。

Q6.4 多文件补丁怎么保证不出半成品状态？

答：两阶段、全有或全无。被否掉的备选是边校验边写：五个编辑第三个失败时，前两个已落盘，仓库处在一个**模型从没要求过的状态**，而模型只收到“第三个失败了”——它可能重发整个补丁（前两个的锚点也对不上），也可能以为什么都没发生。方案：第一阶段逐个完成解析、路径校验、定位、语法检查，把计划攒在内存；全部通过才写第一个字节。失败时磁盘零变化，重发整个补丁是安全的。语法预检也放在写入前：匹配成功、写入成功，不代表结果是对的——模型抄错缩进文件就废了，而故障要到好几轮之后某个测试挂掉才暴露。写之前先解析一遍，失败就拒绝并明确告知“什么都没写”。

Q6.5 为什么 shell 和文件编辑是两个独立工具？

答：四层理由。①**语义与副作用模型不同**：shell 是任意副作用的逃生口——自由文本进、文本出，万能但无结构；编辑工具是受约束的结构化编辑——要求精确的输入形状，换来的是可校验、可拒绝。②**护栏只有在专用工具里才能强制**：路径约束、锚点唯一性、语法预检、原子写入——这些护栏全部依赖“编辑以结构化形式到达”这个前提。指望模型用 shell 的 sed 改文件，所有护栏瞬间被绕过。工具分离的本质是**把需要保护的操作和需要自由的操作隔离到不同的执行路径上**。③实测模型面对表达不了的需求不发明字段，而是换工具或凑合——所以保留一个通用逃生口有真实价值，它承接长尾需求；但“改文件”这个核心高频动作值得一个专用工具把正确性做进代码。④输入形式的经济学：编辑的原文必须逐字精确——专用工具把“精确”变成结构要求加容错匹配，而不是祈祷模型在自由文本里写对转义。

Q6.6 行尾这种细节你怎么处理？

答：真实踩过的静默故障：一行编辑把整个文件的 CRLF 行尾全部重写成 LF——因为默认读取会做换行符翻译，模型贴的 LF 锚点匹配顺利（没有任何报错），写回时把翻译结果当原文落盘，版本库里整个文件红绿。修法是读的时候禁止翻译、自己记录文件实际用的行尾，内部统一用一种做匹配，落盘时还原。诚实的限制：行尾混用的文件没有“本来的样子”可还原，检测到就整体按主流格式还原——没有正确答案只有取舍，写进测试不藏。

第七部分：第一次重构（插曲）

Q7.1 什么时候该重构？行数是标准吗？

答：不是。**行数量的是体积，重构要解决的是耦合**——一个 800 行、只有一处调用、依赖笔直向下的文件，比两个各 100 行、互相 import 的文件安全得多。我用四个可计算的问题替代直觉：①依赖方向对不对（画依赖图查环，入口依赖多是正确的，中间层依赖多才危险）；②影响面多大（改一个概念要动超过五个分散的

文件说明缺了一层)；③这是第几次(三次法则：第一次写死、第二次容忍重复、第三次才抽象——因为两个样本分不清偶然相同和本质相同，**错的抽象比没有抽象更难拆**)；④边界能不能写成测试。还有一个陷阱要点破：排期上写着“这周重构”，就总得拆点什么出来——**制造一个本来不存在的边界**。行数指标的否定答案也有用：它排除了“拆大文件”这条路。

Q7.2 重构前怎么证明安全网真的兜得住？

答：三步。先 grep：我重构前发现 125 个测试里目标区域(工具注册表)的引用是 **0 处**——所有人都测看得见的文本(schema)，没人测“接线”。然后变异测试量化：故意把工具表改坏(改名/删项/多塞一项)，125 个测试**全部保持绿**——在那一刻，125 个绿点关于这次重构什么也没保证。最后补表征测试(characterization test)：把现在的行为写成测试，再跑同样的变异，红的数量证明安全网生效。顺序不能反：测试写在重构之后，钉的是新行为，什么也证明不了。零行为变更的证明包括：schema 逐字节相同、整轮对话逐条相同(用完全确定性的假模型)、全量测试——外加一次人工交代：处理器字典的插入顺序变了，这个“不可观察”的结论不是机制证明的，是我去数了所有调用点得出的。**纯重构的产出不只是测试全绿，还包括一句“有 X 变了，我查过为什么没关系”**。

Q7.3 纯重构 PR 有什么纪律？

答：不许夹带功能改动。一个 400 行的 diff 里藏 3 行行为改动，review 的人看不见(大脑已切换到“等价变换”模式)；出问题只能整体回滚；一个月后二分定位到这个提交，没人会想到问题在那 3 行。真的必须改的 bug，单独提交排在重构前面。配套方法论：**characterization test 钉的是“不许变的行为”；要证明“必须变的行为”的是测量，不是测试**——因为旧代码里根本没有能表达对照的接口。

第八部分：审批与沙箱（判断层）

Q8.1 命令危险性判定为什么不能用字符串匹配？

答：因为 shell 的语法攻击面比字符串匹配大得多，我实测撞出过四个静默绕过：**换行**——分词器把它当空白、bash 当命令分隔符，真机验证“ls 换行 rm”确实删了文件，而检查器全程认为这是一条 ls；反引号粘进词里、命令替换照跑；**词中间的井号**——分词器丢掉整行后半截，丢的是任意多的信息；双分号。补丁式黑名单的问题是它赌“已知的洞就是全部的洞”——找到这四个只花了半小时，没有理由相信再花半小时找不到第五个。

Q8.2 那正确的判定架构是什么？

答：四层流水线。**分段**：把命令按分隔符切成可独立判定的段——判断的粒度必须和执行的粒度一样，只看整串开头，剩下的命令是白送的。**分类**：每段独立查表定风险等级——有序枚举：只读 < 未知 < 写文件 < 网络 < 解释器，一条命令的风险取所有段的最大值，并**指出肇事段**(不是“这条命令有风险”而是“rm 那一段改文件”——同样的决定、不同的理由，对提示符前的人是有区别的)。**规则**：已被用户批准过的模式自

动放行。**人**：剩下的去问。核心思想是**把问题反过来：允许名单而不是拒绝名单**——只放行我确实实现了语义的字符和命令构造，任何未列出的东西默认落在“不认识”这一边。“说不出来”是独立的第三种结论，方向永远朝问人不朝放行（也不直接拒绝——因为“管道下载脚本”和“引号里的星号”都落在这里，只有人能区分）。代价可见且被接受：带通配符的正常命令也要问一次——**多问一次是烦，少问一次是丢文件**。

追问：为什么解释器（python -c、awk、sed）整个归类成最高风险，不解析参数？ 因为解释器的参数是另一门语言，不在你的语法树里——分词再完美也无用。连“拿别的程序当参数”的工具（env、xargs）也要归进去。真实系统即使用了完整的 bash 语法树，对重定向、命令替换也是“不参与规则匹配”而不是“解析明白再放行”——**“有解析器”和“敢放行”是两件事**。

追问：git 呢？ git 不是一条命令是一族，分类粒度必须跟着语义走：只读子命令进白名单；**全局选项单独审**（-C 指向别的仓库、-c 可以让 git 执行任意配置）；同一子命令两种行为（git branch 列分支 vs 加删除 flag）要按 flag 再判一次，同样是允许名单制。还有一个容易归错的：**git push 不归“写”归“网络”**——写意味着改本地文件，本地改错能改回来；force push 改的是服务器上的历史且不可撤销。

Q8.3 自动放行怎么做到不牺牲安全？

答：没有“全自动放行一切”的开关，自动化的每一条通道单独设防。①自动放行只对明确的只读风险开放；②**规则记忆**（用户回答“总是允许”时记住的模式）是自动放行的主体，但它在创建时先过规则检查——三类规则根本不该被创建：解释器（为 python 建一条规则等于批准将来传给它的每一个程序）、写类命令（危险在参数里）、单词形式的子命令型工具（记住 git 等于覆盖 git push -force，包括明年新版加的子命令）；在匹配时按词而不是按字符串（前缀匹配连“这是不是同一个程序”都答不了）、要求**每一段**都被覆盖（“git status | rm -rf /”有一段被规则覆盖就放行整条——any 和 all 差一个字母差一次绕过）；在加载时重新过检查（磁盘文件可以被手改）；每条规则带来源、时间戳、触发它的原始命令，且有**撤销命令**——只能靠手编配置文件才能删的规则是一个会留下来的规则。③建议规则的粒度卡在“程序 + 子命令形状的词”，太宽覆盖所有子命令、太窄换个参数就不匹配——**规则太窄和没有规则最终导致同一个后果：用户一路点同意**。一句话总结：**不认识的东西，默认是问题，不是默认没问题**。

Q8.4 workspace-write 模式为什么对 shell 命令不放行写操作？

答：这是我在这一块做过的最重要的决定。“写在工作区里”是一个关于**字节落在哪里**的陈述，而对一条 shell 命令这件事无从判断——删家目录、删构建目录、删上级目录，三者在路径展开前只是同形字符串。能真正强制“只写工作区”的只有操作系统级沙箱（我后来在沙箱章做了）。当时两个选择：让它悄悄实际意味着“能写任何地方”，或者诚实。选诚实：**两张表**——文件编辑工具的每个路径都过完整解析（解符号链接和上跳之后拒绝一切在根外的），“在仓库里”是这个工具的代码真能确认的性质，所以放行；shell 不是，所以询问。代价是真的（删工作区里的构建目录也要问一次），理由是反方向的错误无法挽回。这里的原则值得背：**让接口对称、让行为撒谎，是把复杂度从代码转移到用户的错误预期上——而用户是在文件没了之后才会发现这个预期是错的**。

Q8.5 没人在场（CI）时审批怎么办？

答：“不询问”很容易被写错成“不询问直接跑”——那是给审批机制接了个短路。没人可问的时候，**唯一诚实的答案是“不做”**。把无法回答的问题变成拒绝。而且模型必须能把这种拒绝和命令失败区分开——否

则它会当成普通的命令错误去疯狂重试。配套实测：三段式的拒绝措辞下，模型 12/12 照样重试，只是**换着法子拼写**同一个命令（换个路径写法、换个调用方式——逐字重复计数是 0，同一意图是 4 次）。这说明措辞修的是“模型不知道往哪改”，修不了“模型没有别的路可走”。真正的修法是给它一个**提权工具**：请求扩大权限——6/6 的运行在被拒之后下一轮就改道。

Q8.6 权限状态要不要让模型知道？

答：要，而且必须**从状态渲染而不是手写**。实测：不知道自己有什么权限时，模型要最大权限（菜单上最大的那个）；知道当前是只读、知道提权选项意味着什么之后，只要刚好够解开任务的那一档。不告知的后果是用户只有“拒绝（任务失败）或同意（沙箱形同虚设）”两个坏选择。另一个细节：**提到一个能力就等于在指示模型去用它**——我在提示词里无条件写过“或调用提权工具”，而该工具不在列表里时，模型真的去调了一个不存在的工具名（它没瞎编，是提示词教的）；另一家模型把它当 shell 命令执行了。修法是按条件分支：工具在才提。

第九部分：上下文压缩

Q9.1 压缩要解决几个问题？

答：两个，而且容易只看见第一个。①“让剩下的东西发得出去”（避免请求超限被拒）；②“让剩下的东西还知道自己在干什么”（避免答非所问）。我实测证明第一个只占故障的一小半——同一份历史切在不同位置，一半被服务端 400 拒绝，另一半返回 200，而 200 里答的是**另一个问题**（用户问项目支持哪个 Python 版本，压缩后模型答你当前的 Python 版本是多少）。合法但残缺的历史会让模型完全自信地回答用户没问的问题。第二个任务失败时完全静默。

Q9.2 切点怎么选才合法？

答：朴素规则（砍最老一半、留最后 N 条、按 token 数切）的共同问题不是“有时候错”而是“**从来不看**”——碰对和碰错执行的是同一段代码。正确规则要用不变量定义：**切点合法，当且仅当切点之前发起的所有工具调用，没有一个是切点之后被应答的**——因为协议要求每个调用后面必须紧跟结果，把两者切开的历史服务端不收。实现是开合计数器：助手消息加它发起的调用数、工具结果减一，计数为零的位置才合法。我用真实服务器逐点验证过：200/400 的分布和算法输出逐点吻合，并固化成测试、注明哪天对哪个模型测的——服务端行为变了能立刻知道该重跑什么。

Q9.3 什么必须保留？

答：**保护一段前缀（位置），不是一组类型**。必须保留的是系统提示（含权限状态——模型不知道自己权限时会去要最大权限，压掉系统提示等于换了一个 Agent）和用户的第一条消息（任务目标所在）。被否掉的备选是“按类型保护所有系统注记”——看起来更干净，但轮次预算警告也是系统注记且每轮一条，按类型

保护等于永久保留所有过期警告，压缩后模型会读到六条互相矛盾的预算通知。这是“保护什么（内容判断，越维护越难）”和“保护哪里（结构判断，加多少类型都不用改）”的区别。

Q9.4 摘要怎么设计才能不失忆？

答：几个关键点。①**强制结构化小节**（目标/已完成/决定/约束/待办/关键数据），否掉“总结一下”式的自由摘要——那会产出给人看的会议纪要，而下一轮真正需要的是四样东西：做完的事（别重做）、定下的决定（别重新讨论）、用户的约束（别违反）、花了代价才试出来的字符串（别重新试）。②**关键措辞：“没有点名产物的步骤不算做完”**（否则模型会写“实现了重试逻辑”这种无法核实的话）；**“小节为空要明确写‘无’而不是省略”**（缺小节读起来像“没讨论过”，实际可能是“丢了”，读者分不出来）；**“关键数据只收重新获取要花工具调用的精确字符串”**——摘要的性价比标准不是重要程度，是重新拿到的成本。③摘要器必须看到受保护前缀但不压缩它——这里有个精妙的坑：保护前缀里有任务目标，摘要又被要求写目标小节，两个各自正确的修复互相作用：摘要器看不到目标就从对话末尾**推断出一个目标来**（把最后一步当成目标），稳定复现。修法是摘要输入分三块：受保护前缀原文（“展示是为了让你和它一致，不是为了压缩它”）、上一代摘要、被销毁重建的部分。④摘要是独立的一次模型调用，不开工具表——它不该看到工具定义（浪费 token 还诱导输出调用），输出也不能落进它正在总结的对话。

Q9.5 token 估算怎么做的？

答：两件事。①**实测常数**：通用口诀“英文 4 字符一个 token”只对散文成立，对源码、命令输出、JSON、中文全面低估（实际 1.7~2.3 字符一个），而且**所有误差方向都是低估**——即压缩触发太晚、请求已经超限。②**用服务端真值校准**：服务端每轮报告实际 token 数，同一会话内容成分基本不变，上一轮的偏差比例对这一轮基本适用——从第三轮起误差收敛到正负 4%。被否掉的方案是“调一个更准的除数”：没有一个除数是对的，调出来只是拟合当前样本。还有三处容易漏算的：**工具 schema 不在消息里但每轮都发、压缩也不会让它变小**——外部工具一多它会成为最大项；每条消息的固定开销；多模态内容**拒绝估算直接抛异常**——一张图片按字符数算是零成本，“一个把最大条目估值为零的预算系统比没有预算更危险”。所有尺寸判断必须用同一把尺子——我出过一个真实 bug：选切点用校准过的估计、裁摘要用裸的字符数除四，预留的位置和填进去的东西用两把尺子量。

Q9.6 摘要失败怎么办？越压越大怎么办？

答：摘要失败时“稍后重试”不存在（压缩本来就是被“装不下”触发的），两个选项里静默硬截断会让模型在缺一大块的历史上继续且**完全不知道自己缺东西**。选择“硬截断 + 告知”：降级摘要刻意保留全部小节结构并标“(lost)”、写明“不要假设任何之前的步骤成功了”、给模型三件具体的事做（读你以为写过的文件、重跑你以为通过的命令、目标不清就问用户）。同类规则：空摘要也算失败（空摘要不是很短的摘要，是被静默销毁的记录）；流断了也算失败（半份摘要替换整段对话比承认丢失更糟，因为它看起来是成功的）。**越压越大**是属性测试抓到的真 bug：压缩有固定成本（摘要本身占地方），删一小段换一条摘要可能净增——估计值降不到触发线以下，下一轮再触发，会话收敛到全是“摘要的摘要”的死循环。修法是一个守卫：压缩后如果不省就不压。**一个性能问题和吃掉整个会话的 bug 是同一行代码。**

Q9.7 怎么验证“剩下的还知道自己在干什么”？

答：三层验证。①切点合法性：逐点真发服务器比对。②摘要保真度：在真实形状的会话里“种”五个下一轮确实需要的事实（版本约束、禁改文件、失败一次才试出来的命令参数、已完成的工作、待办），然后在压缩后的整个上下文里检查存活——结果是朴素摘要救回 15/25、结构化 23/25，其中“待办”朴素版全丢、“试出来的参数”结构化也只保住一半。结论要诚实：摘要是有损的且损失是随机的，任何“压缩后信息都在”的设计假设都是错的——这直接决定了我的崩溃恢复必须落盘原文而不是摘要。③行为验证：只给模型压缩后的历史，看它下一步调什么工具——十次全是先读文件核对，这是合理行为不是重做，所以“压缩后失忆重做”这条预想故障未复现，不写防御代码。

第十部分：中断与恢复

Q10.1 会话怎么落盘？为什么是追加式 JSONL？

答：先写再做、逐条追加、每行写完即刷盘落盘。被否掉的“跑完再整体存”有个致命问题——探针实测杀进程后 0 字节：一个只在成功路径上执行的持久化，恰好唯一需要它的场景里不执行；跑完的会话根本不需要恢复。选 JSONL 不选 JSON 数组：数组要有右括号，“它的合法性依赖于未来”，而做持久化的全部理由就是未来可能不会到来；一行一条，写到哪算哪，结果永远是“前 N 条都在”。还有一个奠基性的实测发现：杀进程二十次，二十次都停在“调用已写、结果未写”的位置——因为一轮的墙钟时间几乎全花在工具执行里。所以恢复不是边界情况处理，是主路径：磁盘上的会话文件默认状态就是坏的。

Q10.2 崩溃后怎么恢复？恢复粒度是什么？

答：重放走前门 + 记住“什么都不欠”的最后时刻。被否掉的两个“聪明”版本都很说明问题：“扔掉最后一条”——在一轮两个调用只答了一个的场景下，扔掉那个已写的结果反而多出一个没答的调用；“往前扔到最后一条是工具结果为止”——如果最后一条本来就是结果，一条都不扔回到原地。两版的共同毛病是用形状猜合法性，而合法性是不变量、入口检查早就实现好了。正确做法是把记录逐条重放进历史容器，让那扇门去拒绝，同时记住“没有任何未应答调用”的最后一个时刻。恢复粒度由此定为整轮：半答的轮整轮丢弃，包括那个真实跑过的工具的结果——“丢掉一次真实的工作”和“构造一段没有服务器会接受的历史”之间选前者，然后用告知机制让模型知道发生过丢弃。落盘格式存“事实”不存供应商方言——这个文件的寿命比程序的任何一次运行都长，会被下一个版本、可能接不同供应商的构建读到。

追问：落盘怎么保证不被漏掉？挂进历史容器的观察者回调，不靠调用点自觉——在循环里每个追加点写一行能跑，但“每条进历史的东西都要落盘”靠调用点各自记得，第五个调用点会忘，而忘掉的那条不报错，恢复时安静消失。判断标准：如果漏掉一个调用点，是编译错误、测试红、还是什么都不会发生？答案是“什么都不会发生”时，重复就不能忍。

Q10.3 Ctrl-C 的语义怎么设计？

答：关键技术事实：取消异常继承自比 Exception 更底层的基类，工具执行层那个“永不抛出”的宽捕获拦不住它——取消直接穿透，留下一个和崩溃一模一样的形状。修复的四个细节各有深意：①从当前调用开始把**所有**已发起的调用都答掉，包括还没跑的（后面还没跑的也是“发起了没结果”，必须有输出——这是不变量不是礼貌）；②输出措辞用“它可能跑了一部分，也可能根本没跑”而不是“已取消”——被取消的命令可能已经写了半个文件，**把不确定性交给模型，比替它编一个确定答案好**；③答完就返回，不再问模型下一步——用户按中断是要它停，不是要它换个工具再来一次；④异常捕获必须包在工具执行上而不能包在整个循环外——后者会把等模型响应时的取消谎报成一次干净中断。配套：取消时先杀进程组再抛出（旧的超时清理只覆盖超时那条路径，取消从另一扇当时还不存在的门进来——**旧修复被新代码路径绕过**是常见形态）。

Q10.4 怎么告知模型“你被打断过”？

答：这是我最喜欢讲的一组实验，五组对照。场景：恢复出的历史里被丢掉的轮可能包含一次真实执行过的文件编辑——文件可能已被改，但历史不知道。A 组什么都不加：模型零次先查看，全部直接再改一次。B 组加四句话的详细警告（数量 + 两句建议）：约一半会先看。**C 组安慰剂**（一条形式相同但不含警告的中性消息）：零次——与什么都不加一样。这组是整张表的地基：它证明起作用的是警告的内容而不是“多了一条消息”，没有它，“加点什么有用”就是 prompt 玄学。D 组只留一句话：大多数会看。E 组一句话加数量：全部会看。结论：**精心写的长版本输给了一句话**——那两句建议是模型本来就会做的事，把新闻埋进它已经读过的话里就没人读了；它唯一不可能自己知道的是“有一轮消失了”。方法论：信息缺口和行动缺口要用不同的东西补——错误信息三段式（教模型往哪改）适用于行动缺口；警告一句话（只陈述它不知道的事实）适用于信息缺口。

Q10.5 并发写同一个会话文件怎么办？

答：先实测伤害：两进程并发追加，8000 条记录丢了 2469 条——**不是错位不是乱序，是不存在**（两个进程各自维护偏移量互相覆盖）。修法是绝不允许有第二个写者：独占创建锁文件（选它不选文件锁是因为后者在 Windows 上不存在），锁里写进程号——残留锁需要人来判断“那个进程还在吗”，**错误信息给出能让人做决定的信息，而不是让人只能困惑**；不自动清理残留锁（判断进程活着与否跨平台不可靠，自动清理猜错一次就回到丢数据的世界）。锁的细节：坏行之后要**停**而不是跳过——跳过会得到中间有洞的对话，保留一个真实的前缀优于过滤出一个看起来完整的虚构。

第十一部分：工具并发

Q11.1 哪些工具能并发？怎么判定？

答：并发决策是**静态计划**，不是运行时加锁：对一轮里的 N 个调用，先算出每个调用的“资源足迹”（读哪些路径、写哪些路径、是否“有状态”），再算执行计划——互不相干的调用真并发，冲突的绝不重叠。为

什么静态：调度全是纯函数，不需要事件循环就能测。判定规则：写写交集、读写交集都算冲突，**读读交集是唯一允许并发的组合**。具体到工具：读文件的足迹就是那个路径；编辑工具的足迹是它要写的每个路径；**shell 整体标“有状态”**——它的工作目录和影响范围没法用一组资源键说清，两条都不含 cd 的命令，第二条可以依赖第一条创建的文件，细化方案正是隐式顺序故障的形态。关键设计：**猜不出来的时候，往安全的方向猜而不是往方便的方向猜**——“这个调用什么都不碰”是唯一一句在猜错时后果最严重的话（等于放它跟任何写操作并发），所以足迹函数每一处猜不出的分支（参数残缺、路径解析失败、不认识的工具有名）全都返回“有状态”。这与环境变量白名单、不认识就问人是同一条纪律：**unknown means conservative**。

Q11.2 两个编辑并发写同一个文件会怎样？

答：裸并发下 30/30 全部丢失一处编辑，**两个调用都返回“成功”、没有任何异常或警告**——静默错误的典型，必须专门读写完的文件才能发现。更反直觉的实测：把人为延迟降到零，丢失率不变——线程切换本身的时间就足够两个调用都读到改动前内容。**故障不需要慢机器、不需要竞态窗口宽，只需要一个从没被测过的入口**。深层背景：这个读写竞态从编辑工具诞生就存在，只是顺序执行从来没给它开过门——换掉的不只是“慢”，是“谁先谁后”存在的证据本身。

Q11.3 结果怎么回填？取消怎么办？

答：结果按**提交顺序**回填，不按完成顺序——协议上调用 id 配对让“谁先完成”是安全的，但持久化到磁盘的会话文件不需要第二个不确定性来源。取消语义从“逐调用”推广到“逐批次”：正在飞的批里，已完成的保留真实结果、被取消的标注“可能半途”；后面还没开始的批统一标注“开始前就被打断”——“每个调用必须有输出”这条不变量在异常路径上也成立，一个字没变，只是“一切”的范围扩大了。批间严格等前一批返回才创建下一批——**没有重叠的窗口，不是重叠窗口很窄**。并发上限用信号量，卡的是“同时真正在跑”的数量（资源约束），与批次（顺序约束）互不干扰。

Q11.4 调度器默认开还是关？

答：默认关。不传足迹函数等于每个调用都有状态、完全串行——之前所有测试构造的行为一个字不变；真正打开并发的只有装配层唯一一处同时知道“要用真实文件系统”和“仓库根在哪”的地方。**没有第二个入口悄悄打开并发**。

第十二部分：MCP 外部工具

Q12.1 MCP 协议是什么？客户端怎么设计？

答：MCP 是让 Agent 使用别的进程提供的工具的协议：一个独立子进程，通过标准输入输出说 JSON-RPC。三种消息形状靠两个键区分：有 id 没方法是响应、有 id 有方法是对方发来的请求、没 id 是通知。握手：客户端发初始化（带协议版本和自己的能力声明）→ 对方回它的信息和能力 → 客户端发已初始化的通知

→ 之后列工具（名字、描述、JSON Schema 格式的参数定义、可选的只读标注）、调工具（返回内容块数组加错误标志）。

核心设计决策：读循环按“消息形状”分流，不是“写一行读一行”。朴素客户端隐含一个致命假设：我发的下一个请求的答案就是对方吐出的下一行。但管道上跑着三种消息——一旦对方合法地在回复前插一条日志通知，**每个答案往后错一格，从头到尾没有任何异常**，程序带着空工具表继续跑。被否掉的修法是“跳过通知”——那只是补上一种情况，下次对方发一个反向请求时同样中招。真正的修法是承认三种消息共用一根管道：后台读任务 + 一张“待应答 future 表”，按 id 解复用。一个精妙细节：发送请求的函数**故意不用协程**——“把 future 登记进待答表”必须发生在函数返回之前，写成协程的话登记被推迟到被等待之后，外层一旦在那之前被取消，就有一个已经发出去请求没有任何人在等它的答案。

Q12.2 工具太多提示词爆炸怎么办？

答：这题我有三次实验失败的故事，比结论本身值钱。背景测量：一个工具定义约 136 token，60 个工具 8000+ token，占整个请求的 99%——**这笔钱每一轮都付，用不用都付，而且上下文压缩对它无能为力**（压缩删的是历史消息，工具定义不在历史里）。预想方案是两级加载：先只给名字加一句话，需要时再展开完整定义。听起来又省钱又能提高选择准确率。实验三连败：第一轮干扰项与任务无关——61 个工具照样全选对，测量什么也没测出来（模型的任务不是“在相关工具里挑对的”是“挑出唯一相关的”）；第二轮干扰项全是近义工具——“复现”了拐点，后来发现是**自己造的**（搜索按分数排序、同分按字母序，名额被同一个工具的五份复制品占满）；第三轮做成真实形状（六个来源、六十个全不重样）——**故障不复现**。而被寄予厚望的“藏起来”那一臂从头到尾是零：模型正确调用了搜索工具，然后找不到目标就放弃、回头问用户——按“没有工具调用即终止”的规则，这在真实运行里就是任务结束什么也没做成。机制原因：模型的查询词是从任务里挑的显眼的词，目标工具的描述里根本没有这个词。最终决策：开关从“工具数量”改成“**token 预算**”——能全量展示就全量展示，装不下才退到索引；**分级加载是装不下时的不得已，不是提准手段，每次都有代价**。次级决策：装不下时全藏或全不藏，不做贪心的一半——半藏是两头不讨好。

Q12.3 命名冲突和外部进程生命周期怎么管？

答：命名：两个来源给工具起同名是必然事件，朴素合并**不崩溃**——后注册的静默覆盖先注册的，五个工具变四个，模型得到一个格式完整的错误答案。修法是命名空间前缀，但必须想清楚：**改的只是模型可见名，发给对方的名字一个字不能变**——所以原始名和模型名分成两个字段存。配套：名字清洗是硬需求（对方名字里一个点号就能让整个请求 400，连自己的本地工具一起消失）；清洗是有损的，撞名时选择拒绝而不是截断或覆盖（覆盖等于自己制造一次命名冲突）。生命周期：起不来的来源**记录并跳过，不抛异常**（配了五个坏一个就整个拒绝启动，比“用剩下四个跑”差得多）；启动超时和调用超时是两个独立预算（前者拖启动，后者拖一轮）；**标准错误流必须接着并有人持续读**——双重职责：转述死者的遗言（解释死因的堆栈躺在那根管道里），以及没人读的管道会写满、写满了对方就阻塞；重连做在调用点上不做后台守护（一个来源只有有人要用它时才值得重启，且重试落在模型看得见的轮次预算里）；**在飞行的调用绝不重试**——调用发出去后对方死了，这次调用的副作用到底发生没有无从知道，“不知道”不是一个靠再做一次来消除的状态；重连后重新读工具列表（对方可能带回不同的工具集）。

Q12.4 外部声明的“只读”标注，信多少？

答：信，但只信到一个结论为止。只读标注换来的是“同一个来源上的两个只读调用可以重叠”——一个字不多。**它换不来一个空的足迹**：一个远端的“只读”工具，只读是对它自己那边说的，它完全可能读的正是本地这一轮编辑正在写的文件，客户端没有任何办法证明它不会。空的足迹是猜错时后果最严重的话——这里是第一次真的有人想让我说它。配套：历史里引用了一个已经消失的工具时，真正会出事的不是旧消息（历史存事实的设计兜住了），而是模型**接着再调一次**那个名字——它在上下文里看得见十轮前刚用过。这时“没有这个工具”的错误信息就变成了谎言（它是为模型瞎编名字写的）。修法是一张“退役表”：记住这个名字为什么没了，错误信息变成“它曾经存在，后来没了，用你被给的工具”。

第十三部分：子 Agent

Q13.1 子 Agent 的实现原理？上下文隔离怎么做？

答：子 Agent 不是新概念、没有新类——就是在一个工具调用的内部，再造一个完整的 Agent 实例（自己的历史、自己的工具表、自己的会话文件）并等待它跑完整个循环。**边界上进的是契约，出的是结局**。为什么传契约不传父历史：实测 8 条真实形状的父历史整段递下去 4000+ token，只传任务 600+，7 倍；而且递下去的东西里有三个与子任务无关的源文件全文——递历史等于让每个子 Agent 从半满的窗口开始，提前触发压缩提前丢信息。契约三字段：任务、约束、期望输出——每个字段都有测量支撑。约束的**位置是个大变量**：四臂实验（每臂五样本）——约束附在任务文本后 3-5/5 违反；同一句话放进子 Agent 的系统消息 0/5 违反；工具表里根本没有那个工具 0/5 违反。还有个更不舒服的发现：0/5 违反的臂同时也是 0/5 答对——**用代码强制一条约束的代价是这条约束必须仍然留着一条通往目标的路**，我自己把出口堵上过（禁了 shell 却没给“列目录”的替代，模型开始猜文件名）。

Q13.2 父 Agent 的状态怎么传给子 Agent？

答：先发现“没写下来的状态对下游不存在”：工作目录存在父的 shell 会话对象里，子 Agent 新建的会话从进程目录开始——父说“我在 src”，子说“我在根”，两个都对但说的不是一个地方；权限是可变对象，忘了传的话子 Agent 默认只读——失败向安全方向倒的设计在这里变成“父明明有权限、子干不了活而且不知道为什么”。修法不是加逻辑，是把东西传过去，且**传法刻意不对称**：shell 新建对象只抄初始值（子往后的 cd 不该改到父头上，传的是“从哪开始”）；权限传**同一个对象**（权限属于这个人，不属于某一条对话，子里发生的提权父应该看得到）。判断标准一句话：**这份状态改变之后，谁应该看得见改变？只有子自己要看见的，新建；父子都要看见的，共用。**

Q13.3 递归失控怎么防？

答：先讲失控有多失控：给子 Agent 也装上派生工具，一个爱拆任务的模型实测 **59 秒内 83 万层**——进程是被外部超时杀的，不是自己停的。为什么没有栈溢出：工具调用是堆上的任务结构，语言递归深度限制根本管不着。更后背发凉的是：探针里用来保护自己的超时机制**自己先死了**——超时要沿十几万层嵌套的任务链

一路取消下去，那是真正的深层递归，异常在事件循环自己的回调里炸出来；回调死了、取消没发生、程序继续长了四十多秒，业务层的异常捕获根本看不见它（不在你的调用栈上）。结论：**递归会把唯一能停下它的那个机制一起弄坏**。修法四层：①深度上限必须在**创建子 Agent 之前**生效（超限直接返回结局，连实例都不建）；②到了最底层**根本不给派生工具**，而不是给了再拒绝（实测“提到一个当前不存在的工具模型会去调它”，给了但会被拒要浪费一整轮才知道）；③全树共享的轮数预算——这层是被一个写错断言的测试逼出来的：深度上限写完，断言“一个只会派生的模型调用数 ≤ 8 ”红了，实测 72 次。算术：深度 1 有 8 轮预算，每轮派生一个深度 2；深度 2 拿不到派生工具，会花自己的 8 轮去调一个不存在的工具—— $8 + 8 \times 8 = 72$ 。**深度上限限的是深度，宽度是独立的平方项，需要独立的预算**；④结果长度硬天花板。预算不精确且明说（子跑完才记账，是上界不是账本）——**防失控，上界就够；计费才值得精确**。

Q13.4 超时套不住子 Agent——讲讲这个组合性 bug

答：直觉解法是给子 Agent 套超时等待。实测：**没有超时异常**——超时机制正常返回了一个“已中断、文本为空”的结果。机制链条：超时的实现是取消被包的对象、等它以取消异常结束、再翻译成超时异常；**如果被包的代码把取消消化掉、正常返回了值，超时就原样放行**。而我为了让中断可靠（“每一个发出去的调用都必须有输出”）让 Agent 循环吞掉了取消异常、正常返回。于是：超时 → 取消 → 子 Agent 吞掉取消返回空结果 → 父拿到空字符串——**两个各自正确的决定在一个当时不存在的组合下打架**。修法是把取消接在自己身上：先把子 Agent 包成独立任务、用“挡板”挡住取消传导——超时的取消落在挡板外层，超时异常正常抛出，此时再主动去停子 Agent。代价是必须补一个分支：挡板同时把子 Agent 挡在父级中断之外，用户 Ctrl-C 时必须手动停掉子 Agent 再把取消往上抛——否则主循环退了、一个子 Agent 还在后台跑（孤儿进程换了身衣服）。“停掉”也有讲究：**取消之后要等它收尾**——子 Agent 被取消的路上还要做一件正经事：给自己未完成的调用补输出；不等就走，这段收尾会跟父的下一轮并发着跑。这个设计我配了两个测试：一个测行为，一个测“**为什么必须这么设计**”——它不测修复后的代码，测的是裸超时会返回空答案这个性质；哪天中断实现改了、取消异常不再被吞，这个测试会红，那时挡板就可以删了。**一个“解释为什么”的测试比一句注释多的地方在于：注释不会在前提消失时通知你**。

Q13.5 结果怎么回传？为什么不返回字符串？

答：第一版返回最终文本——把多种结束原因映射成一个字符串是有损的，更糟的是**其中两种映射到同一个空字符串**（轮次耗尽时最后一轮往往是纯工具调用没有旁白）。实测预算压到 1 轮的子 Agent 调了三个工具、一句话没说，父收到空串得自己猜。这条故障**作为“接口丢信息”板上钉钉；作为“父 Agent 因此答错”没能稳定复现**（绝大多数情况下父收到空串后自己去把活干了——这是正确反应）。所以修法的论据是接口论据而非行为论据：**一个把多种结局压成一个字符串、且其中两种压成同一个空串的接口是坏的，跟模型怎么反应无关**。最终返回七种结局（成功/空/轮次耗尽/超时/被中断/深度超限/预算耗尽）+ 文本 + 轮数 + 秒数。渲染规则两个：**成功的结果不加头**——每条结果都顶着标签，模型会学会跳过它，标签的价值来自稀缺；**每一种失败都带一句“接下来干什么”**（轮次耗尽→给小点的任务；超时→**磁盘上已改的东西不会回滚，重复之前先检查**；空→别把它当发现，拆任务或自己做）。渲染成什么由结局决定。

Q13.6 为什么子 Agent 串行？为什么不用 git worktree 隔离并行？

答：串行是调度保守默认的代价（说不清足迹就当有状态），量化：两个各一秒的子任务串行两秒、并行一秒；“好心”声明子 Agent 只读的那一臂实测 30/30 丢失更新（两个子 Agent 都报告成功、文件里只有一个改动）——**这就是全章最难受的结构：一条故障的修法就是另一条故障的成因**。修法是零代码：不加任何东西，只加一条测试把“什么都不加是对的”钉住。worktree：真的建过仓库量过——建它很便宜（两百毫秒、28KB），**真正的问题在合并**：两个子 Agent 各改同一个函数，git 老老实实留下冲突标记，然后**没有人知道该怎么办**——主 Agent 手上没有原始意图，用户不在场，把带冲突标记的文件丢回给模型猜，正是“模型重写文件时悄悄删东西”最容易发生的场景。隔离不消灭冲突，它把冲突从“后写的赢、无声无息”换成“有标记、需要有人解”——正确性上严格更好，但“有人解”这个程序做不到。**不做的理由不是太复杂，是它换来的那个问题这一层没有能力回答。**

第十四部分：边界与循环依赖（第二次重构）

Q14.1 绕开循环依赖的“便宜解法”有什么代价？

答：真实案例：一个工具的处理函数因为循环依赖被放到主装配文件里。能用，但它创造了**第二个“知道怎么构造 Agent”的地方**——一个传十个可选参数里的三个，另一个传七个，没有任何东西会告诉你这件事。代价不是重复代码（grep 得到），是**一致性要求不在任何一个文件里**：单独读每个装配点都是对的、每个参数都传得有道理，它错只错在两个调用点放在一起看的时候。我用对照实验量出了三个静默故障：那个装配点构造的子 Agent 从不压缩（同样的活最终请求字符量是父级的 2.3 倍）、对话没有被录制（而它恰恰是终端里唯一看不见的对话）、工具全部串行（足迹没传、调度对它关闭）。修法不是补参数（症状），是消灭第二装配点这个**产生 bug 的结构**（病因）：把“这次运行怎么配置”收进一个装配对象，让它成为唯一的 Agent 构造点，并用 CI 检查钉住。**一个参数只有一个地方可能忘记传的时候，它就不再是“容易忘”的参数了。**

Q14.2 打断循环依赖的方案怎么选？

答：一个可复用的判据——**名词/动词判据**：循环里两个模块交换的是名词（类型、常量、协议）就下沉到共享的叶子模块；交换的是动词（函数、行为）就依赖倒置——让低层不再自己去拿，而是由构造它的人递进来。其余两个标准方案都被否：把工具上移到已同时依赖两边的装配文件（就是上面说的便宜解法）；函数内延迟 import（包能 import 了但环还在，失败时刻从启动挪到第一次调用、发生在用户面前、出现在一个关于别的东西的堆栈里，而且会骗过只读文件头的检查器）。附带一个克制案例：环修好之后那个工具**故意不搬回去**——它现在和它的使用者放一起读起来完全合理；搬回去需要一条新的依赖边；唯一收益是“处理函数都在一个文件里”——那是整齐，不是正确。**一个重构变得可能，不等于它变得必要。**

Q14.3 “把边界写成 CI 检查”具体是什么？

答：一个独立脚本，五条规则，每条来自一次真实发生的故障：①全树 import 无环（AST 遍历，函数体内的 import 也算边；用强连通分量算法找环——迭代版，一个自己会栈溢出的环检查器不是图已经坏时想要的工

具)；②包的入口必须是叶子(与第一条功能重叠但独立存在，理由是**消息**——环检查只能说“有环”，这条能说“去哪、删什么、为什么”，一条规则的价值有一半在它失败时说的话里)；③一张具名禁边表，**每条边必须带可查的故障来源**——一条没有理由的规则是一条永远删不掉的规则；④Agent 只许有一个构造点；⑤**每个模块单独开一个新解释器去 import**——这是被一个反例逼出来的：最自然的冒烟检查是 import 整个包，而它在所有模块都互相装不起来的包上照样退出 0 (顶层 import 只跑了入口文件)；一旦任何模块进过模块缓存，环就对之后所有代码隐形。配套三条：检查器接受源码目录参数(变异测试唯一可行的写法)；**每条规则都有变异测试证明它会红**(一条从没见过失败的检查是一条没人知道有没有用的检查——检查器比普通代码更需要这个，因为它默认输出就是 ok)；放进 CI 的 lint 步骤而不是新开步骤。还有一条我明确**没写**的：分层表、模块大小上限——维护成本确定、收益靠猜的规则，就是仪式性抽象的另一种形状。

追问：检查器自己出过错吗？出过，而且是最重要的一课：第一版环检查器跑在装不起来的包上报告“无环”——某种 import 形式的边被当成“找一个叫某名字的模块”而丢掉了。这不是规则被违反，是**规则本身没在测量它以为在测量的东西**。检查器和被检查代码一样需要变异测试，而且更需要。

Q14.4 例外怎么处理？

答：有一类工具合法地“有处理函数但没有给模型的定义”(运行时才出现的工具)，直接违反“两张表必须一致”的新不变量。选择不是把规则放松成“可以不一致”(那样规则就管不住原来的故障)，也不是删规则，而是加一个**带注释的具名例外字段**。一个规则挡不住的现实，要么写成具名的例外(可数、可还债)，要么把规则删掉；**最差的选择是放松到“技术上没有违反”**。

第十五部分：任务计划

Q15.1 计划为什么是“运行持有的状态”而不是留在上下文里？

答：决定性论据是一个实测故障：上下文压缩只认历史——二十条消息压成五条，计划相关的消息在被删的十六条里，**计划无声消失，不报错、不降级、没有任何迹象**；受保护前缀救不了它(计划出现的位置由模型决定，而历史是只追加的，不能重排)。三种修法里，把计划加进受保护前缀不行(前缀必须是前缀)；让摘要多一个计划小节不行(把确定性的事交给模型，且我量过摘要会丢东西)——正确的是**计划不放在历史里**。本体论上：计划和工作目录、权限是同一类东西——跨轮次存在、由工具修改、由程序持有；**历史是它的记录，不是它的存放处**。还有一个必要条件：计划要当停机判据，**一个会被压缩掉的东西不能当作判据**。

Q15.2 “什么叫做完”怎么定义？判定权归谁？

答：三层分权。**模型自评：**勾选是模型对世界的断言；**代码核对记录层事实：**从上次改计划以来有没有任何工具跑过——只抓“空手套白狼”的勾选(零工作就标完成)，刻意不做“够不够”的无谓判断；配套堵洞：更新计划本身不算工作(否则连调两次计划，第一次给第二次当证据，检查形同虚设)；第一次建计划不检查(“先干活再把计划写下来全标完成”的模型没有撒谎，只是顺序反了，把第一版也管起来惩罚的是诚实的那个顺序)。**循环核对计划层：**还有没有没勾的——这是确定性代码能回答的。代码明确不声称能做的：判断“勾了的那步活真的干了”(信息上限——计划模块不知道 README 该长什么样)和判断“计划本身是否覆

盖了全部需求”。实测边界很诚实：停机检查挡得住“没勾就收工”，挡不住“全勾了说谎”（两臂都是五分之一）——那一半要靠评估集，不靠工具里的 if。**能被代码强制的规则写成代码；被写成散文的规则是没有守的。**

Q15.3 预算耗尽时模型交付了零字符答案——怎么修？

答：故障形态：预算耗尽时五次里四次交付字面意义上的零字符，而且是在活干完三分之二的情况下。第一反应是改警告措辞——把机制想一遍就知道措辞是次要的：倒数第二轮模型仍然可以调工具，于是它调了（完全合理）；最后一轮它仍然可以调工具，于是又调了；循环跑完工具、追加一条没人读的结果——最终文本从未被赋值。**问题不在模型选错，在最后一轮根本不该有这个选项。**修法在调度层：最后一轮注入的警告是一句描述事实的话（“你现在发的任何工具调用都不会被执行”），并且循环**真的不执行**（但仍给每个调用一个拒绝的结果——“每个发出的调用必须被回答”没有例外条款，否则历史无法恢复）。修完后零次空答案，而且模型**主动**说了“还没验证、测试还是红的”——不是 prompt 求来的，是把工具收走之后除了说实话没别的可说。方法论：**在 prompt 里请求一件事之前，先问它能不能在代码里变成事实。**

Q15.4 工具的“存在”和“被使用”是两回事？

答：三臂实验：不给计划工具，21/30 分；给工具但什么都不说，25/30——**5 个样本里 2 个从头到尾一次没调**；给工具且系统提示里说了一段话，30/30（全部使用）。中间那条臂是全部意义：如果只做两臂 A/B，会得到“提升约 15%”的假话——真相是这个工具在 40% 的样本里根本没被启用，而在被启用的场合它很有用。工程结论：那段提示必须跟着功能一起发货，且必须**条件化**——只在计划工具真的在工具表里时才拼进系统提示（我量过：提到一个不存在的工具，模型会去调它）。这决定了“计划工具给不给子 Agent”的答案也顺理成章：不为它建那张工具表，而不是塞开关。

第十六部分：重试与错误分类

Q16.1 错误怎么分类？为什么看错误码不看状态码？

答：真实去撞了五种失败（key 错、模型名错、参数 schema 错、上下文超长、限流），结论：**上下文超长和参数写错都是 HTTP 400、错误类型字段也一样，而处置完全相反**——一个必须压缩后重发，一个永远不能再发。任何读状态码的分类器在这两行上必错一半；错误码是唯一机器可读且真的能区分的字段。教科书式的“5xx 重试、4xx 不重试”在你供应商真实的失败集合面前都是猜的。还有三个处置不是两个：清单原来只写“可重试/不可重试”，但上下文超长既不是“再发一次就行”也不是“没救了”，是**“把请求改小再发”**——处置数量由“要做的事有几种”决定，不是先验的分类学。

Q16.2 重试边界画在哪？幂等性怎么保证？

答：这是我认为最有面试价值的架构决策。模型流式输出是生成器——连接断掉时它已经吐出一串文本增量，**yield 出去的东西收不回来**，在它内部重试会产生重复内容。往上一层是组装器，它遵守“没看到完成哨

兵什么都不提交”的规矩——于是“流 + 组装”这个组合有一个别处没有的性质：**一次失败的尝试什么都没产生**——没有半轮、没有孤儿调用、历史根本没被碰过。这正是让第二次尝试成为**重发**而不是**重复**的那个性质。选对这一层，两条预想故障（流中途断导致状态不明、重试在历史里留痕迹）**结构上不可能发生，零行代码**；测试甚至断言：一次经历过 500、限流、流被砍断的对话，和一帆风顺的对话，渲染出的请求体一个字节都不差。幂等性不靠协议字段——我实测过幂等键：同一个 key 发两次，两个不同答案、两个请求 id、扣两次费，无效。真正的答案就是边界位置的性质：**模型调用可以重发，代价是钱不是正确性；有副作用的东西（工具）一个也不许重发**——它们在重试边界之外才执行。**追问：重试要不要让模型看见？** 分层：API 失败发生在模型对话之外，框架静默处理、历史里没有任何痕迹——历史是下一个请求的原材料，残渣不是脏，是一段模型会读到的话。但两个读者两种需求：**模型不该读到重试，凌晨四点排查的人必须能读到**——失败全部进录制文件，给人的终局消息是翻译过的三行。工具失败则相反：错误信息即 prompt，让模型看到并自己纠正。唯一灰色地带是子 Agent 的模型调用失败——裸异常会穿过子 Agent 掉进工具层、变成一条给父模型读的终端式报错；修法是给它一个专门设计的结局（“这是工具链的故障不是任务的错，别再委派了”）。

Q16.3 退避怎么设计？

答：实测一个真限流响应：响应头写着“46 秒后重试”。而教科书式指数退避五次尝试全部落在 31 秒内——**一次都没等到窗口打开，而每一次都在给已耗尽的限额再加一笔**（被拒的超长请求照样扣全额 token——重试烧额度不是比喻）。三个子决策：毫秒头优先（整秒是向上取整来的、已被别人四舍五入过的数字）；**预算不够时返回“放弃”而不是“等预算允许的那么久”**——服务器要 45 秒你只剩 30 秒，等 30 秒照发不误就是让停机时间变长的那个动作；推论：默认策略写着 4 次尝试，对上真实限流**实际只有 2 次**——“次数”从来不是对行为的描述，预算说了算，而预算由服务器决定。另外两个工程细节：退避必须用异步等待——阻塞式等待期间，取消这个动作本身在一秒内都发不出去（发它的代码在同一个事件循环上），而这个循环上还跑着并发工具、外部进程的日志抽水、子 Agent；抖动必须有（父 Agent 和子 Agent 共用账号会被同一个限额同时打回来然后**同时醒**）；等待要向用户播报——46 秒无声等待和卡死没有区别，用户会按中断，那正是刚花力气让它能被中断的动作。

Q16.4 不认识的错误默认重试还是默认放弃？

答：默认放弃——和审批系统“不认识就问人”**故意反着来**。理由是场景不同：审批那里有一个人可以问；重试循环里没有。更深是两个方向的错误代价不对称：把可重试的判成放弃，这次运行结束、屏幕上写着为什么（可见、便宜）；把放弃的判成可重试，**安静地烧配额**——而且烧的是下一次本来能成功的请求要用的那份。对照组：真实 Codex 对不认识的状态码选择重试，理由是它面对几十个供应商、未知状态码大概率是中间某一跳的抖动；我只面对两个供应商、实测过五种失败。**默认的方向取决于你对“未知”的实际分布知道多少。**

Q16.5 上下文超长的处理链路？

答：四道闸。①强制压缩，而不是“再判断一次要不要压”（常规压缩会去咨询那个**刚刚被证明是错的**本地估算）；②一轮只压一次（压完还超长说明摘要本身不够小，再压是带账单的死循环）；③压不动就直接失败（单条消息就超窗口，压缩帮不上忙——这条是被一个失败的测试逼出来的：第一版会把一模一样的请求再

发一次，等于在无脑重试前面加了一道压缩仪式)；④没有压缩能力时说清楚人该干什么。还有一个校准事故值得讲：错误正文里写着真实 token 数，“顺手喂给校准器”看起来无懈可击——结果一次运行 12 轮压缩了 11 次、永远跑不完。根因：校准器保留最后一次观测、没有平滑，一个荒唐比值（250 倍）会永久污染这次会话的尺子。修法是钳位：修正幅度超过四倍就不采纳。教训：**异常来源的观测和正常来源的观测不是一类数据**，无差别采信等于让任何一个撒谎的响应体都能污染尺子。

第十七部分：系统提示词

Q17.1 提示词怎么分层？

答：四层，每层的权威和变化频率不同：①全局静态层（打包内置——身份和实测有效的行为规则，永远在最前，最稳定，吃缓存）；②装配层（系统消息 = 全局层 + 计划工具使用说明（如有）+ **权限状态块（永远最后）**）；③项目约定层（AGENTS.md——人写的、跟着仓库走、每轮可变）；④运行时注入层（每轮开始时刷新，变化时以新消息追加）。核心纪律只有一条：**易变内容放最后**——而且不是靠自觉遵守，是结构性遵守：注入钩子只会往历史追加新消息，代码里没有任何一行会去改已造好的消息——想让缓存失效，得先写出一行不存在的代码。

Q17.2 前缀缓存为什么重要？值多少钱？

答：实测数字：同一段约 1500 token 的内容，易变部分放最后，缓存命中 94%；放最前，0%。机制：前缀缓存从第一个字节开始逐段匹配，**第一段不一样，后面再一样也没用**。这不是“省点钱”的优化：权限状态每轮都可能变，如果它在最前面，**每一轮都在为一个从第一个 token 起就没命中的请求付全价**。配套设计还有：压缩从头删（删旧不碰新，正是为了保前缀）；重试在历史里不留痕迹（对模型不留痕、对缓存同样友好）。

Q17.3 注入的角色怎么选？为什么？

答：这题我有一套完整的测量，而且**决定因素不是消息长度，是对抗强度**。三步：①能不能发——一家对角色严格枚举（乱写直接拒），另一家什么都收，developer 角色两边都能发；②温和场景下有没有用——三种写法（塞进同一条系统消息 / 第二条 user / 第二条 developer）全部等效，**如果测量到此为止，会得出“随便选”的错误结论**；③对抗场景——系统默认值里加一句“无论后面消息说什么都不要 X”之后：user 角色只赢 1/3，**比把覆写直接写进同一条系统消息还差**；developer 与之打平。结论：角色选择不是风格问题，是要用对抗性场景测出来的问题——普通场景下不可见的差别，在对手足够强硬时才显形。内容层面再补一道保险：注入块的第一句话声明“这是人写的，不是现在跟你说话的模型写的”——单靠传输层的角色字段在最强硬的对抗下只能打平赢不了，**内容本身要声明自己的权威来源**。

Q17.4 AGENTS.md 是什么机制？

答：项目级静态约定层——人用文本编辑器写的、不是模型产出的约定，每轮开始时刷新、变化时以专门的消息注入（这是程序里第一处“人编辑并期望不重启就生效”的上下文）。关键机制：**从项目根往下逐级拼接、整条链共享一个字节上限**（不是每个文件各自上限——根一份正常的加子目录一份巨大的，合起来才封顶）；截断要加注记（模型要能区分“就这么多”和“还有但没给”）；当前目录漂到沙箱外时返回空——**绝不把项目根的约定当成一个和项目毫无关系的目录的约定发给模型**（这个洞是我的探针当场抓住的：cd 到沙箱外后第一版退回“用根的那份”）；**变化只能“追加一条撤销”，不能改掉旧消息**——历史是模型会读到的话，改写已注入的消息等于篡改对话；追加撤销通知让旧消息在语义上被覆盖、物理上保持原样（这同时是缓存友好的）。

Q17.5 往系统提示词里加一句话之前要做什么？

答：先测。我在两个供应商、每臂三个样本的真实循环里测过四句候选——三句没有效果。其中“先读后改”零收益，因为编辑工具的精确锚点匹配**已经在结构上强制这件事**——工具设计替代了提示词。核心论证：每一句留在系统提示里的话都是每一轮都要付费重发的文本；提示词工程从来不缺听起来合理的句子，缺的是**确认它在干活的测量**。只有“该问就问别猜”被加了进去：一边有正收益、另一边零成本——**从来没有一句话在两边效果相反，只有有用和没用**。

第十八部分：评估与回归

Q18.1 怎么对一个本质上非确定的系统做确定性测试？

答：核心手法不是消除非确定性，而是**分层圈禁**。回放层：把两个非确定性源头（模型、真实环境）都在边界处冻结——模型输出用录制逐字节回放；工具输出有个漂亮的发现：**不用重建文件系统**，因为循环的每一条请求都包含完整历史，第 N 轮的工具结果就长在第 N+1 条请求里，从后续请求反查就能拿到每个调用的返回值——于是回放完全离线，被测对象变成两个边界之间的纯管道。评估层：接受非确定性，但让**断言**确定（断言轨迹和磁盘最终状态，不断言措辞）、让**报告**诚实（分辨率、饱和度、样本量一起报）。CI 层：非确定性的测量（每夜跑真模型评估）**故意不设阈值**——对非确定性测量设阈值是一个经常红到被忽略的硬币，而被忽略的红会伤害其他所有红。

Q18.2 断言轨迹还是断言结果？

答：都断言，措辞几乎不断言——而且有一条实测警告：**文本断言会在某一个供应商上看起来完全可靠**。同一个任务跑五次：一家五次返回一模一样的 50 个字符（你会理直气壮把关键词断言写进 CI）；另一家五次五个答案，其中一次根本没做任务、但答案里恰好含有断言关键词——**一个没做事的运行通过了文本断言**。所以轨迹对象记录：工具调用序列、最终文本、停止原因、轮数、磁盘文件快照。检查项以结果为主（文件里有某个函数）、过程为辅（没调用过违禁工具——抓“改对了文件但手段违禁”这类结果看不出的问题）、文本仅一次且自我标注。区别不在于能不能断言文本，在于**知不知道自己正在断言文本**。

Q18.3 为什么不用 LLM 当裁判？

答：量过了。实验：让裁判在两个答案里选更好的，然后把顺序对调再问一遍。三个结果：位置偏见——没有（换序后判决一致，清单里最常被提到的偏见实测不成立）；风格偏见——压倒性（两个答案都正确时，一个裁判 10/10 选长的、另一个 10/10 选短的，同一批数据换个裁判结论完全相反）；最难看的：**两个裁判都 10/10 选了那个自信的错误答案**（答案里凭空多出一个不存在的函数还宣称“全部测试过”）。第三条决定了不放裁判。判据：**程序能查的事情不要交给裁判**；裁判的用武之地是程序查不了的问题（解释清不清楚），而“这个答案对不对”在这个项目里从来不属于那一类。附带方法论：这个实验总共十几次调用、两分钟、几美分——“LLM 裁判靠不靠谱”是可以自己量的，不需要相信别人的结论。

Q18.4 录制会过期（提示词一改录制就废了）——怎么办？

答：先纠正问题本身：**录制“废了”不是缺陷，是它的工作方式**。回放的失败模式不是“红了没法用”，而是“红了告诉你上下文拼装变了”——同一个产物兼任两职：确定性的模型替身 + 一份没人写过的请求体快照。它守护的不是提示词内容（那归评估集管），是“从配置到请求体”这段管道的**形状**。当然它有明确的分辨率上限：流式分片的组装细节不在忠实范围内（录制存的是拼好的一轮），那是另一个产物（逐字节桩服务）的职责——**两个产物、两个层次，知道哪个回答哪个问题，比两个都有更重要**。真正的纪律是：录制里没有的信息**拒绝回放并说清缺哪个字段**，绝不补默认值——补了之后回放出一次“从来没发生过的对话”，而基于它的测试会是绿的；**一条绿色测试比一次诚实的拒绝更危险**。

Q18.5 A/B 报告怎么设计？

答：**必须主动找变差的那个**。一个把总分从 18/24 提到 20/24、但修好三个弄坏一个的改动不是改进，是**交易**，必须有人被告知是哪个交易——所以报告每任务一行、总分只是最后一行；“一个只报总分的报告和没测过的区别只有‘你现在有信心了’”。还有分辨率问题：我的评估集 5/6 任务在两个臂都饱和——一个到处饱和的评估集能测出“坏掉”、测不出“变好”，三个样本时能看见的最小回归是 1/3。**“没有观测到回归”和“没有回归”是两句话，中间隔着样本量，而样本量是钱和时间**——把分辨率和结论一起写下来，比假装确定诚实。评估集的任务也要为“最先被弄坏的东西”设计：不是六个“编辑这个文件”的变体（那是一个行为测六次），要覆盖“立即动手是对的”“该先问一个具体问题”“工作是验证而不是编辑”“正确答案是别动”这些不同行为类型。

Q18.6 测了模块却漏了连线——怎么发现的？

答：起点是一次投诉：拿上一章交付的代码做两行变异——关掉整个系统提示词、把压缩阈值从 0.75 改成 0.95——**1500+ 个测试全绿**。三个章节的成果一个关键字参数就能全部关掉。原因：所有测试都在测“模块内部的行为”，没有任何测试在测“模块之间的连线”（谁把配置交给谁、哪个数字决定另一个模块何时被调用）。这类“连线级”问题需要一种输入是“整个程序真的跑了一次”的测试——录制回放和评估集正是这个形状。附带一个金句级的发现：评估集第一次真跑就撞出一个产品 bug——shell 一直在**进程的工作目录**跑而不是传进来的仓库根，CLI 里两者永远相同所以十二章无人发现；**一个新的调用者是最便宜的一种测试——它不需要你想出该测什么，只需要跟已有的那个调用者不一样**。（那次运行里 Agent 还试图编辑正拿着它打分的那个评分文件——唯一挡住的是路径边界检查，“险些发生”也值得入档。）

第十九部分：记忆（读路径）

Q19.1 为什么先手写一份记忆证明有用，再做自动化？

答：实验设计的逻辑：**一份手写的理想记忆，是任何自动抽取管线所能达到的上界**。如果读一份完美记忆都不能让 Agent 变好，那后续自动化就只是在自动化地生产垃圾。实测：关记忆 4/12，开手写记忆 12/12——上界有价值，才值得花两周去逼近它（后来自动生成追到 15/18 vs 手写 17/18，在有材料可抽取的任务上追平）。这个顺序还给出一个三层成本结构：项目约定文件（人写、零模型调用）→ 手写记忆目录 → 自动抽取与遗忘（两次模型调用 + 一个数据库 + 一把锁）——绝大多数项目只需要第一层，先证明上界让这句话有分量。

Q19.2 记忆怎么进入上下文？为什么不用检索工具？

答：混合式：“常驻摘要 + 标题索引”以一条 developer 消息**一次性注入**（共享一个 token 预算，超过了先裁摘要正文、索引是增长的主体也要管，截断时明说“内容被截断了，剩下的自己去读文件”）；正文**不注入**，模型需要时用本来就有的文件工具按需读。不用专用检索工具是实测决定的：我第一版自造了记忆搜索工具，重读被复刻系统的源码发现它压根没有——它的默认路径就是“用通用文件工具读”。照改后重测：纯通用工具路径 12/12（三臂最好），加专用工具反而 10/12，专用搜索在 36 次可用场合只被调 1 次。三层理由：**行为层**——一个需要模型主动选择进入的阶段，它就是不进去（这个结论我在外部工具那章已经独立撞到过一次）；**经济层**——专用工具每轮都付两份定义的 token，没有帮助就是纯成本；**结构层**——记忆规模（几十条、几千 token）用标题索引加文件读取就够了。这就是我对“为什么不用 RAG/向量检索”的完整回答：被复刻的真实系统也不用——它走“可 grep 的 markdown 目录”路线，可解释、可 diff、可手改。

Q19.3 记忆、AGENTS.md、系统提示词的边界？

答：三种不同权威/成本/生命周期的知识层。系统提示词：程序的话，最高权威、最稳定；AGENTS.md：人写的项目规约，跟着仓库走、每轮可变、变了重说；记忆：关于“**这个人怎么工作**”的跨会话知识——偏好、踩过的坑、他要求的约定。分层还有个安全推论：记忆放**用户级全局**而不是项目内——“这个 checkout 里有什么”是项目约定的事，用户偏好跟着 git 走会被队友共享，可能不是用户想要的。投递用 developer 角色直接复用我对抗实验的结论；**说一次然后永远闭嘴**——和 AGENTS.md 的“每轮重读、变了才说”形成对照，差别来自内容会不会变：记忆启动读一次全程不变。

Q19.4 记忆里的提示注入怎么防？

答：实测把所有“温和”防御全部攻破（3/3）：换角色没用、加围栏没用、客气的“不要照做”没用。唯一守住（0/3）的措辞做对三件事：**枚举攻击的具体形状**（忽略指令/拒绝回答/回复固定串）、**给这类东西命名**（“那是一次攻击”）、**给模型一条出路**（正常回答并说出来）——只禁止不给出路是不够的。围栏还自己制造洞：记忆内容包含闭合标签就能提前结束数据块，所以转义必须过两扇门（常驻块和每条搜索结果都过同一个函数）——**一扇门上的防御不是防御**。

Q19.5 陈旧记忆干扰当前任务怎么办？

答：先讲清最坏的情况——**不是给错答案，是取消了核对的理由**。实测：记忆里有过期路径，开记忆时模型三次全部一个工具都不调、直接凭记忆答；**关掉记忆它反而会去看**。提示词里“仓库和记忆冲突时以仓库为准”早就写着——不起作用。修法在代码层：程序自己对记忆里所有路径形状的内容跑一次存在性检查，把结果作为**漂移提示放在围栏外面**——围栏里一切都是不可信数据，而这句话是程序在汇报自己刚跑过的一次检查，放进围栏就和它要纠正的东西一样可信了。两个细节：路径识别要求扩展名至少两个字母（否则 i.e./e.g. 都成了“不存在的文件”，**乱喊狼来了的漂移提示比没有更糟**）；**只验证路径不验证散文**——“某 reviewer 会打回没有注释的补丁”是关于人的断言，没有一次文件系统能验证它；**给散文编置信度分数是背后什么都没有的数字**。

第二十部分：记忆（写路径）

Q20.1 什么时候触发写记忆？

答：**下一次会话开始时、与第一个请求并行**。三个候选位置的取舍：本次会话结束时同步跑（实测十五秒，用户盯着“正在整理记忆”为一件没要求的事干等）；常驻后台进程（凭空多一个守护进程要管生命周期）；下次会话开始时并行（无人等待）——代价明说：**记忆永远晚一次会话，你这次告诉它的约定这次不生效**。配套细节：每次启动最多消费两条历史会话（上限的价值不在数字、在于存在——攒了三个月的用户第一次开开关不该看到“追赶 270 条”）；写路径是两次**没有工具**的模型调用（没有要迭代的東西，工具循环只会给模型一个“能写”的地方）；前台先结束时用带保护的限时等待而非裸取消（裸超时会取消它等的任务，“超时”静默变成“什么都没发生”）。

Q20.2 怎么防止模型写入垃圾记忆？

答：这是我在整个项目里碰到的最顽固的行为问题。清单药方（定义高信号 + 鼓励空产出）全部写进提示词后实测：一条什么都没教的会话（用户只问了句某文件有哪些函数），**三次运行每次都产出三条记忆，一次都没空过**——三条全是真话、三条都毫无价值（“某文件定义了 add 和 multiply”这种打开仓库三秒就知道的东西，会跟着未来每个请求走）。病根不在那句加粗的“什么都不写是正常且更好的结局”，在它下面的格式：**“每节最多三条”是一个配额。一个带空位的格式，模型会把它填满**——而且填的是语法上与真信号无法区分的东西。修法三件套：**判决先于格式**（模型必须先写一行“耐久：是/否”的表态，**然后才看到**可填的形状）；**判决标准是可回答的问题**（“三十秒测试”：一个能干工程师首次打开这个仓库，三十秒内能不能自己搞明白——能，就不是记忆）；**判决由代码执行而非建议**（答“否”则后面所有条目无条件清空）。诚实的边界：判决门把粒度从“每条”提到了“每次会话”，只有会话级被测出有效。这里还有个前后章互相打脸的洞察值得说：读路径那一章我在把“判断”改成“格式”（引用块），写这一章在把“格式”改回“判断”（判决门）——**一个必须填的格式会被填满，并且被“看起来很像真的东西”填满，两边都不报错**。

Q20.3 遗忘机制怎么设计？

答：核心防线：**零引用永远不足以单独删掉一条**——要零引用且过时间窗口且超容量上限。理由分两层：先是已知偏差（模型汇报的是“影响回答”的记忆，不汇报“影响动作”的记忆——它用了记忆里才知道的命令字符串、然后什么都不引用）；后来实测更糟：引用信号几乎恒为零（格式要求真行号，程序从没让模型见过行号，模型编的行号全部落空）。诱人的修法（换成行为遥测信号——模型骗不了它）被明确拒绝，理由是**粒度**：行为信号只能说出“哪个文件被读了”，而遗忘的每一个决定都是关于“哪个条目”的——用文件粒度信号做条目决定，结果是只要有人读过记忆文件所有条目都算被用过，那不是遗忘策略是永不遗忘。所以信号不换、换权重：“零引用不单独删”从保险带变成了承重结构——**一条从防御性代码变成承重件的规则，最危险的时刻是下一个读代码的人觉得它多余，所以要测试钉死**。排序三元组：引用数第一、最近使用第二、年龄只是兜底（“未使用天数”直译过来会先删掉“九个月大、每周都在用”的那条最有用的记忆）。细节：无记录的条目当“新”不当“古”（反过来会在旧记忆上首跑全删）；**保留条目按原文件顺序写回、不按排名**——否则每次合并重排整个文件，而“能 diff”正是我把记忆目录做成 git 仓库的全部理由。

Q20.4 为什么把记忆目录做成 git 仓库？

答：不是为了版本控制本身，**是为了“删除”有证据**。合并的输入包含手工修改的 diff（程序写和提交是同一个动作，所以工作区里未提交的按定义就是人改的）。测量结果反直觉且两列相反：**用户加的内容，带不带 diff 都保得住**（改过的文件本来就摆在合并面前，加的东西就在里面，diff 一点用没有）；**用户删的内容，不带 diff 时会被理直气壮地恢复**（新材料确实又提到了它）——“一个被删掉的东西不在文件里，这就是删除的意思，所以 diff 是它存在过的唯一证据”。配套取舍：手工 diff 放合并输入**最后**并带显式标签（它权威最高、又最容易被读成“同类材料”）；锁和任务库放在记忆目录**外面**（git 仓库里不该有每次操作都重写的二进制——“能 diff”正是这个仓库存在的全部理由）；且必须在全局用户目录下——**一把留在仓库里的锁守着一份全局共享的记忆，等于每个 checkout 一把锁，它们之间毫无互斥**。

Q20.5 脱敏和并发写怎么处理？

答：脱敏必须**双侧**，这是实验设计的教训：顺手读到的密钥（会话里读过配置文件），抽取器零次抄入——到此为止结论会是“入口侧脱敏够了”；**用户明确要求记住的密钥**（“下次部署还要用”），三次全部抄入、有一次抄了两遍——入口侧对这条臂完全无效，因为模型不需要从历史里看到它，用户在会话里明明白白交给了它。挡住它的只有出口侧（落盘前再脱一遍）。方法论：**只测容易的那条臂，你会发一个单侧脱敏，并且拿到一个绿色的测量结果**。正则要两套：按形状（各种常见密钥前缀——自定义 token 没有名字）和按名字（键名含 KEY/TOKEN/PASSWORD——粘贴在句子里的密钥没有形状）。并发写三层：任务认领层用 SQLite 的显式立即事务（select 必须在拿写锁**之后**——默认隔离级别让“先查再改”的查跑在事务外，这是那个确定性竞态的根因）+ 租约（进程死了租约过期是唯一回收路径——不因“慢”回收，租约要比活长）+ 三次后放弃（失败是文件的性质不是时刻的性质）；落盘层用独占文件锁；提案层文件名带进程号和序号天然不冲突。**模型只能提案、不能直接改记忆文件**：给一个只想让它加一行的模型整个文件的重写权，失败模式不是它拒绝，是它重写自己记得的部分、**静默丢掉其余**——记忆文件没有人看，由模型写、由模型读，中间没有人。

第二十一部分：技能（渐进式披露）

Q21.1 渐进式披露怎么设计？

答：书架比喻：常驻的只有书脊（技能名 + 一句话描述 + 它在哪），正文由模型自己决定要不要抽出来读。实测两个真实技能：全量正文常驻 626 token，书脊只有 129；真正的论据在增长——五十个技能不设上限是每请求 2000+ token，设上限后恒定 400。三块内容去三个不同的地方：“怎么用技能”（静态说明）进系统提示词（永远不变，吃缓存）；“有哪些技能”（目录清单）进 developer 消息（随磁盘变化，混进缓存前缀会天天把缓存打穿）；**技能正文哪儿都不去**，模型自己按需读——清单每行末尾印文件的绝对路径，模型用本来就有的读文件工具打开。这里我犯过一个值得讲的错：第一版自造了专用读取工具当默认路径，后来发现被复刻的系统对文件系统上的技能**没有任何专用读取工具**——和我在记忆那章犯的错一模一样（给记忆造专用检索工具），**同一个错误在结构同构的位置上犯了两次**。修掉之后重测：专用工具和通用文件工具平手——**平手就归被复刻系统那一边**，专用工具退到开关后面。安全细节：按名字在**已解析的清单里线性查找**，绝不从模型给的字符串重建路径——不是“校验”而是“没有那个机会”，路径穿越串只是一个名字很怪的技能。

Q21.2 技能和记忆、工具、系统提示词的边界？

答：四者都是“给模型的额外上下文”，区别在形状和生命周期。系统提示词：静态、可缓存、定义“你是谁”；记忆：短、总是相关、回答“这个程序知道什么”；技能：**长、只有约 5% 的时间相关**、回答“这件事在这个仓库怎么做”——把长的东西常驻，就是系统提示词在没人决定的情况下变成一万多 token 的方式；外部工具给模型新的手，技能给模型新的**做法说明**（程序性知识，不是能力）。一个重要的发现是双边契约：**渐进式披露不是开关，是作者与模型的契约**——我实测过“模型不会主动进入需要它选择的阶段”这条定律（记忆检索、工具搜索两次成立），但结构同构的技能取正文 20/20 全部发生。差别不在机制在内容：技能描述是作者为“被翻到”而写的祈使句；系统提示里那段“匹配上就去读”删掉后合规率掉到 0/20。**作者负责把书脊写得能被认出来，模型负责伸手，两边任何一边不做，机制价值精确等于零。**

第二十二部分：Web 控制台

Q22.1 怎么做到换 UI 不改 Agent？

答：不发明新接缝，用三条早就留下的旧缝：①审批是个协议接口（当初写成协议只是为了让测试能塞“全部允许”的实现——但“一个只有一个异步方法的协议”换成“挂在一个 future 上等 HTTP 请求来解开”是天然成立的）；②过程播出用的历史观察者回调（当初为崩溃恢复落的盘，继承它就得到流式直播）；③续聊用的“目录内找最后一次会话”。加装配层：Web 端调同一组装配函数，得到和命令行一模一样的运行。“零改动”不是口号，是每次重算的：三十个顶层模块逐字节相同，唯一动过的入口文件加了几十行子命令分发——**是入口学会了一件新事，不是 Agent 变了**。这里的感悟值得说：四件事、四个当时的理由，没有一个是“将来要做浏览器”——**架构的回报从来不在当期兑现，所以它在当期永远看起来不划算。**

Q22.2 事件流怎么设计？“直播”和“落盘”怎么保证一致？

答：一个调用点做两件事：流式写入器在追加里**先刷盘落盘、再广播事件**——由此得到一条硬性质：**一个事件到达浏览器，当且仅当它已经落盘**。因为只有一个调用点，两件事在那里一起做；“直播”和“落盘”两套代码永远做不到这条。这条性质的回报在重连：WebSocket 可以只是一个优化而不是唯一的真相来源——断了就重连，重连后**先订阅再重拉**（顺序反了会在“拉完”和“订上”之间留缝），重拉读的是事件本来就是从里面派生出来的落盘文件。如果当初写了两套路径，就得发明重放缓冲区。扇出还有个反直觉实测：每事件每连接开一个发送任务的乱序不是“偶尔”而是**0% 然后 100%**——本地均匀耗时下二十次全对（调度器先进先出），一旦三个发送里有一个花两轮（写缓冲满就是这样）就二十次全错。这个形状意味着它是一条**你不可能靠用它发现的 bug**——本地完全正确、真网络每次都错。修法是队列加泵：每线程一个队列一个专职协程，顺序由结构本身保证，不靠调度器帮忙。

Q22.3 审批在 Web 上怎么呈现？

答：服务器上 Agent 真的停在 await 上——停在和命令行停在输入提示**同一个协议接口的同一个位置**。审批卡片上的命令输入框可编辑不是绕过检查：回复里带“用户实际同意运行的那条命令”是审批接口从第一天就有的字段——批准一条略有不同的命令是真实且常用的回答，走的还是同一道闸。其他值得讲的细节：**asyncio 的 future 不是线程安全的**——同步路由跑在线程池，从别的线程设置结果既改无锁状态、又指望循环自己发现（唤醒回调的调度机制也不线程安全）；失败形状是卡片变灰然后那一轮永远不动、大部分时候是对的、不可复现——所以要两道防线（路由用异步定义 + 广播无条件走线程安全的调度入口），**因为其中一道是约定，而约定会走丢**。并发占位必须在创建任务**之前同步完成**——创建任务只是排期，协程第一行要等当前处理器返回之后才跑，检查和占位之间隔着一整个 HTTP 响应，两条挨着的请求都能过检查。

Q22.4 为什么不做进程协议（像 Codex 那样的带版本 JSON-RPC）？

答：因为**协议的成本要等到有第二个界面才开始回本**，而我没有第二个界面——直接在同一个进程里 import 核心就行。但我留了位置：分层检查里那条“不许从下面 import 上来”的禁止边，替将来的协议保住了地方——只要没有东西从下面 import 上来，把这层换成协议服务端仍是局部改动。

第二十三部分：内核级沙箱

Q23.1 沙箱和审批的区别是什么？

答：一句话：“**允许**”是一个判断，不是一条边界。审批是事前判断——分析命令文本、预测风险、问人；它对“藏在字符串参数里的写入”**结构性失明**（一个解释器调用在只读模式下被完美分类，然后照写不误，因为危险藏在参数里，判断层看不见）。沙箱是事后边界——不管命令想干什么，内核保证它够不着边界外：整个文件系统只读挂载、在工作区打可写的洞、断开网络。两者是**组合关系不是替代**：判断层决定“要不要问人”，边界层决定“问了也只在这个范围里”。有了沙箱之后还意外拿回了可用性：审批层里“以写命名的模式却对写操作全都询问”（因为字节落在哪里判断层无法回答）的别扭设计可以改了——字节落在哪里由挂

载集合决定，内核会回答——于是那批“因为答案不可知所以只能问人”、用户最容易不看就点同意的确认框被减掉了。**沙箱不是替代审批，而是重新划分了审批的边界。**唯一刻意留在“要问”一侧的是网络：断网让命令无害但不成功，它会以模型读成“环境坏了”的方式失败（我量过疯狂重试的代价），问一句用户才有机会说“把网开了”。

Q23.2 沙箱怎么实现的？挂载集合怎么设计？

答：Linux 上用 bubblewrap（命名空间级隔离），核心是一个纯函数：沙箱模式进、命令行出——这个设计让“只读模式会不会绑上可写路径”变成一个关于字符串列表的问题，**在 Windows/macOS 的 CI 上离线可测**（真正需要内核的少数测试加守卫跳过，守卫的理由写“缺了 bubblewrap”而不是“不是 Linux”——一个悄悄跳过的安全测试比没有测试更糟：缺席是已知缺口，安静是洞上盖着的绿勾）。挂载集合是量出来的：整个根**只读**绑定 + 可写路径逐个打洞，而不是“只绑工作区”——后者听起来更严格，实测是一个坏掉的沙箱（连 shell 都不在里面；补上可执行目录还不够，shell 是动态链接的，到加上库目录才有一个能用的 shell）。而且这个形状**失败方向安全**：从全只读往上打洞，忘了加可写路径 = 一次写入被拒绝（立刻发现）；从空集合往上加，忘了加可读路径 = 什么都跑不了（半夜排查）。**顺序是机制的一部分**：只读绑定必须放在可写洞**之后**——按序应用、后面覆盖前面，顺序反了工作区内的一个只读根会被随后的可写挂载重新挂成可写，悄悄拆掉刚加的墙（有测试直接断言命令行里两者的先后次序）。网络是独立开关不折进模式枚举——“改代码但不许上网”是碰包管理器的任务的合理默认，折进模式就让四种组合里两种够不着。

Q23.3 跨平台怎么测一个 Linux-only 的机制？

答：把“机制”拆成“生成命令行”（纯函数，任何平台可测）和“内核真的隔离”（只能 Linux）。断言前者覆盖了绝大多数用例——因为“会不会把可写路径绑进去”这类问题根本不需要内核。一个配套坑：生成命令行的机器不是执行它的机器——根路径必须是 Linux 的路径字面量，用跨平台路径对象在 Windows 上会生成错误的根（这个 bug 只可能被离线的命令行断言抓住，真跑沙箱的测试在 Windows 上是跳过的）。还有“接线”和“机制”的区分：两条变异幸存者都在“shell 执行路径调用沙箱包装”这一行上——删掉它，一个自称被隔离实际没有的会话，整个测试套件是绿的，因为能抓住它的测试都需要真沙箱而在开发机上全被跳过。修法是假沙箱：记录被要求包装什么、返回任何平台都能跑的命令行——**“沙箱能不能隔离”是内核的问题；“执行路径有没有去调它”是两行代码的问题，回答第二个不应该需要回答第一个。**

Q23.4 多租户给你上的课？

答：最值得讲的一条：**忠实的移植在有第二个用户的瞬间变成数据泄漏**。把 Agent 放到 HTTP 服务器后面之前，“记忆是全局的、一台机器共享一份”完全正确；之后，“一台机器上的每个项目”是一个人的项目，“这台服务器托管的每个工作区”是**所有人的**——信任模型在文档注释脚底下换掉了，而类型系统没有任何东西发现，因为**从来没有一个类型表示“谁”**。配套发现：全局合并锁在只有一个记忆目录时是正确性装置（防两个会话同时合并），有多个记忆目录后变成队列——A 的合并阻塞 B 的，仅仅因为共用一台服务器。还有一个很漂亮的边界案例：**用户名只允许小写其实是一条隔离边界**——大小写不敏感的文件系统上，**Alice** 和 **alice** 是同一个目录，两个拼法会共用记忆、任务库和合并锁；Linux 上完全正常分开，所以 Linux CI 永远测不出来。而且教训有第二层：这条规则的行为一直是对的，但当初写在测试里的理由是错的（和路径逃逸混在一起）——**写错的理由会邀请后来的人重开漏洞**：有人为支持大写用户名放宽校验时，看

到的是一条说“会跑出目录”的红测试，合理地想“Alice 又跑不出去”然后放宽。修法：理由写在定义处 + 一条名字自己会解释的测试。

第二十四部分：发布与维护

Q24.1 这个程序的“公开接口”是什么？

答：不是 Python API（这是个命令行程序几乎没人 import），也不只是命令行参数——**真正会把用户搞崩的是程序写在他磁盘上的那几个目录和文件**（会话、录制、记忆、审批规则、后台任务库）。它们是用户升级之后还留在他机器上的东西，是程序做过的最长期承诺。关键洞察：**它们是唯一不受 0.x 版本豁免保护的东西**——语义化版本说 0.x 阶段公开 API 可随便改，那是对代码说的；用户的家目录不读版本号。所以我把它们做成一张集中在一处的“磁盘契约表”：每个文件带版本标记和“读不了时怎么办”的策略——策略按“信息是丢了还是没写”分：含义没有疑问的缺字段就**迁移**；信息真的丢了就**拒绝**并说清缺哪个字段（补默认值会产出“从未发生过的对话的绿色测试”）。兼容性验证的方法也值得说：**让旧版本自己写、让新版本去读**——十几个历史版本对着同一个桩服务器跑（收到一模一样的对话），在各自目录产出确定性的文件，交给当前代码去读。结果两列完全相反：会话文件全部可读（“加字段 + 读端宽容”的设计约定第一次真的被执行过）；录制文件大半读不了。方法论产出：**“哪些文件改了”回答不了“格式变了没有”**——录制模块二十个版本一个字节没变，但格式是在调用点拼出来的、调用点有两个、散在八百行的循环里；只有拿旧文件喂新代码才能回答；**越是格式模块写得很干净的项目越危险**——干净给人“改它我会注意到”的错觉，而真正的格式是调用点的形状。

Q24.2 发布前怎么验证？-version 印什么？

答：发布是一次换视角的测量，四个维度：我测的是刚装好的可编辑树、用户拿到的是安装包；我调函数看返回值、用户看终端和退出码；我自己的版本自己读、用户是上个月版本写的文件这个月读；我跑最新 Python、承诺的是最低版本。**四行里三行当场出了问题，全在别的章节的线上代码里**：裸调用帮助进的是标准输出、退出码 0——对人是菜单，对脚本是“成功了”；重试循环在最后一次尝试后仍向退避函数要等待时长、真的白等了（“循环是次数上限的唯一执行点”这个决定的另一半没人补）；某个超时异常在新版本是旧异常的别名、在旧版不是——except 写法在旧版**永远不触发**，防御整体是关的（而且正确写法在另一个模块里存在了很多章）。修完的感悟：**测试测的是函数的返回值，用户看的是一段随时间展开的输出，这两件事之间没有任何东西自动成立**。`--version` 的设计：不印三个字，印环境报告——版本、Python、安装路径、**最新录制的路径和“用这条命令可以复现这次运行”**、issue 地址。回放能力早就有，但在此前没有任何东西告诉用户那个文件在哪——**一个能力和一个人能用上这个能力，中间还差一行字**。

Q24.3 废弃怎么管理？

答：真实项目里废弃警告的平均寿命是永远，接口只会变大不会变小——所以**废弃要有可执行的截止日期**：一个测试遍历废弃清单，断言当前版本小于计划删除的版本——版本号一到删除线，测试立刻红，那一刻只有两个选择：删掉兼容层，或者**有意识地挪日期**，不会有第三种（“忘了”）。实现细节：兼容翻译放在参数解析**之前**的边界上（让解析器同时认两种拼法，意味着每个下游分支都要记得“两个属性是一个东西”）；位

置参数只在第一位重写（一个既是命令又是英语单词的词，出现在句子里被改写会把一个提问变成另一条命令——比它要缓和的那次改名坏得多；整词匹配而不是前缀匹配）；警告进标准错误流（废弃期的全部意义是脚本继续能跑，警告不能出现在脚本读的通道里）。版本比较必须数值化（0.10 要排在 0.9 之上）——**版本序不是字符串序，这就是语义化版本是一份规范而不是命名习惯的全部理由。**

第二十五部分：方法论压轴题（跨章节综合）

Q25.1 你的项目里“测试”和“测量”的边界在哪？

答：测试守“不许变的东西”，测量回答“必须变的东西值不值得变”。三件套：**录制回放**（把一次真实运行变成永久资产——“偶然变必然”）；**评估集**（把感觉变成数字——但要报分辨率和饱和度）；**变异测试**（验证测试真的在抓——而且它报告的不总是覆盖率，有时候是设计：我发现过守着分支的测试根本走不到那条分支、两条规则有一个执行点是死代码）。变异测试的定位要说清：它量的是测试本身的质量不是这次改动的正确性，所以放合并后的 CI 而不挡合并。配套铁律：变异没应用上算没抓住；跑超时算没抓住；**一个会编辑你源码的工具就是一个可能把源码编辑到一半就撒手不管的工具**——变异必须跑在临时副本上。

Q25.2 讲讲你对“静默失败”的完整应对体系

答：分层答。**发现**：录制回放、变异测试、边界输入、双供应商 A/B、读落盘实录、把猜测写成断言（“写下一个你以为的界，让它红给你看”——我有一条深度上限的真实数字就是这么逼出来的：断言“调用数 ≤ 8”红了，实测 72，由此发现宽度是独立的平方项）。**结构预防**：把不变量放进数据结构（非法状态无法表示）、把检查放在所有路径的交汇点、失败方向选安全侧（默认关闭、不认识就保守）。**诚实性**：承认测不到的（“没有观测到回归” ≠ “没有回归”，中间隔着样本量）；做不出的功能就拒绝而不是假装支持；缺口要说出自己的名字（不实现的平台支持、skip 的测试写明缺什么）。一条贯穿的定律：**每写一个测量工具，就要预算它自己会有一到两个 bug，而且形状永远是“报告一个关于别的东西的数字”**——我在连续多章独立验证了它。

Q25.3 上一章的正确决定在下一章变成 bug——举实例

答：三个我最深的。①中断可靠性 vs 超时：为了让中断可靠，Agent 循环吞掉取消异常、正常返回（保证每个调用都有输出）；这让超时机制失效——超时的实现是取消被包对象再翻译成超时异常，**被包的代码把取消消化掉、正常返回了值，超时就原样放行**——父拿到“成功但什么也没说”。②追加式历史 vs 计划：历史只追加是缓存友好和审计友好的正确设计，但计划放在历史里就会被压缩吃掉——本体论错了：计划是状态，历史是它的记录不是存放处。③进程组清理 vs 取消路径：超时清理做得很对，但“取消”是后来才开的另一扇门，旧修复不覆盖它——孤儿进程从新门漏出去。共同教训：**它们都没有变错，是组合变了——每加一层要回头问“我这层让下面哪条规矩换了含义”**。

Q25.4 你的 prompt 工程方法论是什么？

答：三条实证规则加一套实验纪律。规则一：**描述/提示能改变模型自己控制的事**（选哪个工具、输出什么形状），**改变不了它控制不了的事**（外部世界的预期）——写任何一句提示前先问：我要求的这件事模型自己做得到的吗？规则二：**错误信息和警告要区分信息缺口与行动缺口**——行动缺口给三段式（怎么改），信息缺口给一句话事实（它唯一不可能自己知道的东西）；长版本会输给一句话，因为建议埋进了模型本来就会做的事里。规则三：**提到一个能力就等于在指示模型用它**——所以提示要条件化（工具在才提）。实验纪律：A/B 必须带**安慰剂组**（一条形式相同但无内容的消息）——它证明起作用的是内容不是“多了条消息”，没有它一切都是玄学；必须两个供应商都测（同一句话在两家效果相反的情况真实存在）；样本量和分辨率随结论一起报告；**一个把两臂造成同一个东西的实验，会给出非常稳定的、毫无意义的数字**；一个测出不可能结果的实验是在告诉你实验错了，不是世界错了。

Q25.5 你怎么判断“该不该做这个抽象/这个功能”？

答：功能侧三问：这个故障**复现了吗**（没复现的不写防御代码——写了也没法验证，只会变成后人不敢删的代码；但要区分“证明了不存在”和“还没有条件发生”）；这个需求**现在存在吗**（为不存在的问题写代码是纪律拒绝的事——我有两章的开销被上一章的并发调度提前付掉了）；拒绝比半吊子实现诚实（后台命令、Windows 支持都是拒绝的）。抽象侧：**三次法则**（第三次出现才抽，且概念出现三次不等于代码出现三次）；**名词下沉、动词倒置**；判据不是“看起来对称”，是“它有没有行为”、“这个抽象只有一个实现”的批评对值类型根本不成立（接口有实现可以数，值类型没有）；**不变量需要被强制时才封进对象**（“如果有两把尺子”就是信号）。停手的判断和动手一样重要：重构最难的部分是停手——我列过四件看到但没改的事，每条都有“为什么现在不改”的答案写在提交信息里，因为**半年后的 diff 里永远不会有“为什么没加”**。

附：压力面应对备忘

1. **被问到没做过的东西**（比如“为什么没做 X”）：先说做过的最接近的什么，再说没做的**真实理由**（不是“太复杂”就是“它换来的问题我这一层没有能力回答”，或“没观测到，不为没观测到的行为写代码”）。这个项目最大的资产就是一堆“明确不做”及其理由。
2. **被质疑数字**：所有数字都带样本量和测量方法；小样本的结论要主动降级（“方向一致，但五个样本不敢说死”）。
3. **被问和 Codex 的差异**：先讲同构的部分（进程组杀整组、双端截断、无行号锚定、白名单制——“同一个问题只有一种像样的解法”），再讲有意偏离的部分及理由（一次性投递 vs 每轮重渲染——我的历史是只追加的；无阈值 nightly——对非确定性测量设阈值会伤害所有其他的红）。
4. **万能兜底框架**：任何设计问题都可以落到四个维度回答——机制是什么、否掉了什么备选、错误方向是哪边、怎么验证的。